TS

GUÍA INTERACTIVA

TypeScript para React

El lenguaje justo y necesario, y cómo tipar props, estado, eventos, refs, context, hooks y APIs.

Versión PDF · 30 de octubre de 2026
16 capítulos · Los ejemplos interactivos están en la app (npm run dev)

Índice

Empezá acá

1. Cómo usar esta guía

Parte 1 · El lenguaje

2. Tipos básicos e inferencia
3. Objetos: type e interface
4. Uniones y narrowing
5. Funciones
6. Genéricos
7. Utility types y tipos calculados
8. Módulos, tsconfig y declaraciones

Parte 2 · TypeScript en React

9. Componentes y props
10. Estado y eventos
11. Refs y el DOM
12. Context, useReducer y custom hooks
13. Componentes genéricos y patrones de props
14. React 19 con TypeScript · React 19
15. Datos de APIs: unknown y Zod

Parte 3 · Repaso

16. Chuleta y preguntas de entrevista

Empezá acá

Cómo está armada la guía y por qué cada ejemplo está chequeado por el compilador.

Capítulo 1

Cómo usar esta guía

Qué cubre, cómo se chequean los ejemplos y la configuración del proyecto.

Esta guía junta **todo el TypeScript que vas a usar en un proyecto de React**: primero el lenguaje (lo justo y necesario, sin teoría de más) y después cómo se tipa cada pieza de React: props, estado, eventos, refs, context, reducers, hooks propios, componentes genéricos y lo nuevo de React 19.

Está pensada para leerse en orden, pero cada capítulo se sostiene solo. Si ya escribís TypeScript en el día a día, saltá directo a la parte 2.

Los ejemplos se chequean de verdad

Casi todos los snippets de esta guía son **archivos reales** del proyecto, en `src/demos/typescript/ejemplos/`. No están solo para leer: `npm run build` los pasa por el compilador de TypeScript, así que si un ejemplo tuviera un error de tipos, el build fallaría.

Para mostrar los errores que TypeScript te ahorra, los ejemplos usan este comentario:

```
// @ts-expect-error: Type 'number' is not assignable to type
  'string'.
puesto = 42
```

`@ts-expect-error` le dice al compilador "en la línea siguiente **tiene que** haber un error". Si no lo hay, el que falla es el comentario. Así cada error que ves en la guía está comprobado: es exactamente lo que pasaría en tu editor.

> **🧪 Probá esto**
>
> Abrí cualquier archivo de `src/demos/typescript/ejemplos/` en VS Code y borrá un `// @ts-expect-error`: vas a ver aparecer el subrayado rojo con el mensaje real. Pasá el mouse sobre las variables para ver el tipo que infirió TypeScript.

Algunos capítulos tienen además **demos en vivo**: componentes tipados que podés tocar, con su código abajo, igual que en la guía de React.

La configuración del proyecto

El proyecto usa la plantilla de Vite con **TypeScript 7** (el compilador reescrito en Go, unas 10 veces más rápido) y estas opciones, que son las que vas a encontrar en un proyecto nuevo:

Opción	Qué hace
`strict`	Activa todos los chequeos estrictos. En TypeScript 7 viene activado por defecto (en versiones viejas había que ponerlo).
`verbatimModuleSyntax`	Obliga a importar los tipos con `import type`.
`erasableSyntaxOnly`	Prohíbe la sintaxis que genera JavaScript (como `enum`). Lo vemos en el capítulo de módulos.
`noUnusedLocals` / `noUnusedParameters`	Marca variables y parámetros sin usar (en los ejemplos está apagado, porque declaran cosas solo para mostrarlas).
`jsx: "react-jsx"`	El JSX se transforma sin tener que importar React en cada archivo.

Los ejemplos tienen su propia configuración, `tsconfig.ejemplos.json`, que extiende la de la app.

Cómo aprovecharla

1. Leé la explicación y el código. Pasá el mouse por las variables en tu editor para ver qué infiere TypeScript.
2. Romper cosas a propósito es la mejor forma de aprender tipos: cambiá un tipo, sacá un `?`, borrá un `case`, y mirá qué se queja.
3. Los recuadros **En una entrevista** resumen cómo contestar las preguntas típicas. El último capítulo las junta todas.

> ℹ️ **Versiones**
>
> Escrita en octubre de 2026 con TypeScript 7.0, React 19.3 (tipos de `@types/react` 19) y Zod 4.

Parte 1 · El lenguaje

Lo que hay que saber de TypeScript para escribir React con soltura, sin teoría de más.

Capítulo 2

Tipos básicos e inferencia

Primitivos, arrays, tuplas, any, unknown, never y cuándo anotar.

TypeScript es JavaScript con tipos. Los tipos se chequean mientras escribís (en el editor) y al compilar, y **desaparecen** en el JavaScript que llega al navegador: no cambian cómo se ejecuta tu código, solo te avisan antes de que algo se rompa.

Primitivos, arrays y tuplas

src/demos/typescript/ejemplos/basicos/primitivos.ts

```
// Los tipos primitivos van en minúscula: string, number,
  boolean.
let empresa: string = 'Acme'
let salario: number = 3000 // number cubre enteros y decimales
let remoto: boolean = true
let id: bigint = 10n // enteros enormes (poco común en el
  front)
let clave: symbol = Symbol('clave')

// null y undefined son tipos propios. Con "strict" (activado
  por defecto en TS 7)
```

```
// no se pueden asignar a un string: hay que decirlo con una
  unión.
let notas: string | null = null
notas = 'Entrevista el lunes'

// @ts-expect-error: Type 'null' is not assignable to type
  'string'.
empresa = null

// Arrays: dos formas de escribir lo mismo.
const tags: string[] = ['react', 'remoto']
const salarios: Array<number> = [2500, 3000]

// Tuplas: arrays de largo fijo con un tipo por posición (como
  lo que devuelve useState).
const rango: [min: number, max: number] = [2500, 4000]
const [min, max] = rango
```

Algunas cosas para notar:

- Los tipos van en **minúscula** (`string` , no `String`). Los de mayúscula son los objetos envoltorio de JavaScript y casi nunca los vas a querer.
- Con `strict` , `null` y `undefined` **no** entran en cualquier tipo. Si un valor puede faltar, lo decís con una unión: `string | null` . Esto solo ya evita la mayoría de los "Cannot read properties of undefined".
- Las **tuplas** son arrays de largo fijo. Las vas a ver todo el tiempo sin darte cuenta: `useState` devuelve una tupla `[valor, setter]` .

Inferencia: no anotes todo

`src/demos/typescript/ejemplos/basicos/inferencia.ts`

```
// TypeScript infiere el tipo a partir del valor: no hace falta
  anotar todo.
```

```ts
let puesto = 'Frontend' // string
const modalidad = 'remoto' // "remoto": una const no cambia,
  así que el tipo es el literal
const postulaciones = [1, 2, 3] // number[]

// @ts-expect-error: Type 'number' is not assignable to type
  'string'.
puesto = 42

// Las funciones infieren el tipo que devuelven.
function anual(mensual: number) {
  return mensual * 12 // devuelve number
}

// ¿Cuándo anotar? En los "bordes": parámetros, lo que exporta
  un módulo
// y cuando el valor inicial no alcanza para saber el tipo.
let seleccionado: string | null = null // sin la anotación
  sería solo null

// Si declarás sin valor, anotá: si no, queda como any
  implícito y se pierde el chequeo.
let respuesta: string
respuesta = 'Aceptada'
```

TypeScript **infiere** el tipo a partir del valor. Anotar `const nombre: string = 'Acme'` no suma nada. La regla práctica:

Anotá...	Dejá que infiera...
Parámetros de funciones (siempre)	Variables con valor inicial
Props de componentes	Lo que devuelve una función interna
Lo que exporta un módulo, si querés un contrato claro	Callbacks (`map` , `filter` , handlers inline)
Cuando el valor inicial no alcanza (`null` , `[]`)	

Fijate la diferencia entre `let` y `const`: `let puesto = 'Frontend'` es `string` (puede cambiar), pero `const`

`modalidad = 'remoto'` es el **literal** `'remoto'`, porque nunca va a cambiar.

any, unknown, never y void

src/demos/typescript/ejemplos/basicos/any-unknown-never.ts

```
// any apaga el chequeo: todo vale, y los errores aparecen recién en el navegador.
const peligro: any = JSON.parse('{"nombre": "Acme"}')
peligro.noExiste.tampoco() // compila, pero explota al ejecutarse

// unknown es el "any seguro": acepta cualquier valor, pero hay que revisarlo antes de usarlo.
const dato: unknown = JSON.parse('{"nombre": "Acme"}')

// @ts-expect-error: 'dato' is of type 'unknown'.
dato.nombre

if (typeof dato === 'object' && dato !== null && 'nombre' in dato) {
  console.log(dato.nombre) // acá TypeScript ya sabe que tiene "nombre"
}

// never: un valor que no puede existir. Lo devuelve una función que nunca termina bien
// y aparece cuando una unión se queda sin opciones (lo vemos en narrowing).
function fallar(mensaje: string): never {
  throw new Error(mensaje)
}

// void: la función no devuelve nada útil (como un event handler).
function avisar(mensaje: string): void {
  console.log(mensaje)
}
```

Tipo	Qué significa	Cuándo aparece
`any`	"No chequees nada"	`JSON.parse`, `response.json()`, código viejo. Evitalo: contagia todo lo que toca.
`unknown`	"Puede ser cualquier cosa, revisalo antes de usarlo"	Datos externos, el error de un `catch`.
`never`	"Esto no puede pasar"	Funciones que siempre tiran error, uniones sin opciones restantes.
`void`	"No devuelve nada útil"	Funciones y callbacks sin `return`.

> ⚠️ **any es contagioso**
>
> Si una variable es `any`, todo lo que sale de ella también lo es: `peligro.nombre.length` es `any` y TypeScript no te va a avisar nada. Cuando recibas datos de afuera, bajalos a `unknown` y validalos (lo vemos en datos de APIs).

Tipos literales y as const

src/demos/typescript/ejemplos/basicos/literales.ts

```
// Un tipo literal es un valor exacto. Unidos con | arman un
  conjunto cerrado de opciones.
type Estado = 'postulado' | 'entrevista' | 'oferta' |
  'rechazado'

let estado: Estado = 'postulado'
estado = 'oferta'

// @ts-expect-error: '"contratado"' no está entre las opciones
  (y el editor te autocompleta las válidas).
estado = 'contratado'
```

```ts
// as const congela un valor: todo pasa a ser readonly y
  literal.
const ESTADOS = ['postulado', 'entrevista', 'oferta',
  'rechazado'] as const
// typeof ESTADOS es readonly ["postulado", "entrevista",
  "oferta", "rechazado"]

// Y de ahí se puede sacar la unión, sin escribir las opciones
  dos veces:
type EstadoDesdeArray = (typeof ESTADOS)[number] // 'postulado'
  | 'entrevista' | ...

const config = { tema: 'oscuro', columnas: 3 } as const
// @ts-expect-error: Cannot assign to 'tema' because it is a
  read-only property.
config.tema = 'claro'
```

Las **uniones de literales** son la herramienta más útil del día a día: un estado, una variante de botón, una modalidad de trabajo. Reemplazan a los `enum` y el editor te autocompleta las opciones.

`as const` "congela" un valor: todo pasa a ser `readonly` y los strings quedan como literales. Combinado con `typeof` e indexado con `[number]`, te permite escribir la lista de opciones **una sola vez** y sacar el tipo de ahí.

> 🎤 **En una entrevista**
>
> "¿Cuál es la diferencia entre any y unknown?" → "Los dos aceptan cualquier valor, pero any desactiva el chequeo y unknown no: con unknown no puedo usar el valor hasta que lo achico con typeof, instanceof o una validación. Para datos externos uso unknown y valido en el borde."

Capítulo 3

Objetos: type e interface

Opcionales, readonly, extender tipos, Record y la eterna pregunta: ¿type o interface?

Casi todo en una app de React es un objeto: las props, el estado, lo que devuelve la API. TypeScript los describe por su **forma**: qué propiedades tienen y de qué tipo.

type e interface

src/demos/typescript/ejemplos/objetos/type-vs-interface.ts

```ts
// Las dos formas describen la forma de un objeto.
type Empresa = {
  nombre: string
  web?: string // opcional: puede no estar (su tipo es string | undefined)
  readonly id: number // no se puede reasignar
}

interface Postulacion {
  id: number
  empresa: Empresa
  puesto: string
  salario?: number
}

const acme: Empresa = { id: 1, nombre: 'Acme' }
```

```ts
// @ts-expect-error: Cannot assign to 'id' because it is a
  read-only property.
acme.id = 2

// @ts-expect-error: Property 'puesto' is missing in type ...
  but required in type 'Postulacion'.
const incompleta: Postulacion = { id: 1, empresa: acme }

// Los objetos literales no pueden traer propiedades de más
  (excess property check):
// @ts-expect-error: Object literal may only specify known
  properties, and 'pais' does not exist.
const conDeMas: Empresa = { id: 2, nombre: 'Globex', pais: 'PY'
  }
```

- `?` marca una propiedad **opcional**: su tipo pasa a ser `T | undefined`.
- `readonly` impide reasignarla (solo en tiempo de compilación, en runtime sigue siendo un objeto común).
- A un objeto literal no le podés pasar propiedades de más: TypeScript lo marca porque casi siempre es un typo.

> ℹ️ **Tipado estructural**
>
> TypeScript compara **formas**, no nombres. Si un objeto tiene todas las propiedades que pide un tipo, es compatible, aunque nunca lo hayas declarado con ese tipo. Por eso podés pasar un objeto con propiedades de más si viene en una variable (el chequeo de propiedades de más es solo para literales).

Extender tipos

src/demos/typescript/ejemplos/objetos/extender.ts

```ts
// Extender: interface usa extends, type usa intersección (&).
  El resultado es el mismo.
interface Persona {
  nombre: string
  email: string
}

interface Reclutador extends Persona {
  empresa: string
}

type Candidato = Persona & {
  seniority: 'junior' | 'semi' | 'senior'
}

const ana: Reclutador = { nombre: 'Ana', email: 'ana@acme.com',
  empresa: 'Acme' }
const yo: Candidato = { nombre: 'Rick', email: 'rick@mail.com',
  seniority: 'senior' }

// Diferencia 1: solo type puede ser una unión, una tupla o un
  alias de un primitivo.
type Id = string | number
type Par = [string, number]

// Diferencia 2: las interfaces con el mismo nombre se fusionan
  ("declaration merging").
// Sirve para ampliar tipos de librerías (por ejemplo, Window);
  en tu código suele ser una sorpresa.
interface Config {
  tema: string
}
interface Config {
  idioma: string
}
const config: Config = { tema: 'oscuro', idioma: 'es' } //
  necesita las dos
```

¿`type` o `interface`? En la práctica son intercambiables para objetos. Las diferencias reales:

	`type`	`interface`
Objetos	Sí	Sí
Uniones, tuplas, primitivos, tipos calculados	Sí	No
Extender	Con `&`	Con `extends` (da mejores mensajes de error)
Se fusiona si la declarás dos veces	No (da error)	Sí

Elegí uno y sé consistente. Muchos equipos usan **`type` para todo** (porque también sirve para uniones) y reservan `interface` para cuando necesitan extender tipos de librerías. Es lo que hace esta guía.

Objetos como diccionario

`src/demos/typescript/ejemplos/objetos/diccionarios.ts`

```
// Objetos usados como diccionario: claves que no conocés de
  antemano.
type SalariosPorEmpresa = { [empresa: string]: number } //
  index signature
const salarios: SalariosPorEmpresa = { Acme: 3000, Globex: 3500
  }

// Record<Clave, Valor> hace lo mismo y se lee mejor.
const conteo: Record<string, number> = {}
conteo['react'] = 3

// Con claves de una unión, Record obliga a completar todas.
type Estado = 'postulado' | 'entrevista' | 'oferta'
const colores: Record<Estado, string> = {
```

```
  postulado: 'gris',
  entrevista: 'azul',
  oferta: 'verde',
}

// @ts-expect-error: Property 'oferta' is missing.
const incompleto: Record<Estado, string> = { postulado: 'gris',
  entrevista: 'azul' }

// Ojo: con index signatures, acceder a una clave puede dar
  undefined aunque el tipo diga number.
// Activá "noUncheckedIndexedAccess" en tsconfig para que
  TypeScript te obligue a revisarlo.
const deGlobex = salarios['Globex'] // number (o number |
  undefined con esa opción)
```

`Record<K, V>` es la forma de tipar un objeto usado como mapa. Con claves de una unión de literales es muy útil: si agregás una opción a la unión, TypeScript te marca todos los `Record` que quedaron incompletos.

> 💡 **noUncheckedIndexedAccess**
>
> Por defecto, `salarios['Cualquiera']` es `number` aunque la clave no exista. Con `"noUncheckedIndexedAccess": true` en el tsconfig pasa a ser `number | undefined` y te obliga a revisarlo. Lo mismo con `array[i]`. Es más estricto y algo más molesto, pero atrapa bugs reales.

🎤 En una entrevista

"¿type o interface?" → "Para objetos son casi equivalentes. type además sirve para uniones, tuplas y tipos calculados; interface se puede extender con extends y se fusiona si se declara dos veces, que es útil para ampliar tipos de librerías. Uso uno de forma consistente, normalmente type."

Capítulo 4

Uniones y narrowing

Uniones discriminadas, chequeos exhaustivos, type predicates y satisfies.

Una **unión** (`A | B`) dice "es uno de estos tipos". Antes de usar el valor, TypeScript te obliga a averiguar cuál es. Ese proceso se llama **narrowing** (achicar el tipo) y es lo que más vas a hacer con TypeScript en React.

Achicar una unión

src/demos/typescript/ejemplos/narrowing/uniones.ts

```
// Una unión es "uno de estos tipos". Antes de usarla, hay que
  achicarla (narrowing).
function formatearSalario(salario: number | string) {
  // @ts-expect-error: toFixed no existe en string, y salario
    podría ser un string.
  salario.toFixed(0)

  if (typeof salario === 'string') {
    return salario // acá es string
  }
  return `USD ${salario.toFixed(0)}` // y acá, number
}

// Las formas de achicar una unión:
function ejemplos(valor: string | string[] | Date | null |
  undefined) {
  if (valor == null) return // == null descarta null y
    undefined a la vez
```

```
  if (Array.isArray(valor)) return valor.join(', ') // string[]
  if (valor instanceof Date) return valor.toISOString() // Date
  return valor.toUpperCase() // solo queda string
}

type Remoto = { modalidad: 'remoto'; zonaHoraria: string }
type Presencial = { modalidad: 'presencial'; ciudad: string }

// "in" pregunta si el objeto tiene una propiedad.
function donde(trabajo: Remoto | Presencial) {
  if ('ciudad' in trabajo) return trabajo.ciudad
  return trabajo.zonaHoraria
}
```

TypeScript entiende el JavaScript que ya escribís y achica el tipo dentro de cada rama:

Chequeo	Sirve para
`typeof x === 'string'`	Primitivos: string, number, boolean, function...
`x == null` / `x != null`	Descartar `null` y `undefined` a la vez
`if (x)`	Descartar `null`, `undefined` y valores falsy (ojo con `0` y `''`)
`Array.isArray(x)`	Arrays
`x instanceof Date`	Clases (Date, Error, Map...)
`'prop' in x`	Objetos que se distinguen por una propiedad
`x.tipo === 'algo'`	Uniones discriminadas (lo que sigue)

Uniones discriminadas

Es el patrón más importante de esta guía. Cada variante tiene una propiedad literal en común (el **discriminante**: `status`,

`type`, `kind`...) y trae solo los datos que tienen sentido en ese caso.

src/demos/typescript/ejemplos/narrowing/discriminadas.ts

```ts
// Unión discriminada: todas las variantes comparten una propiedad literal (el "discriminante")
// y cada una trae solo los datos que tienen sentido en ese caso.
type Pedido<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string }

function mensaje(pedido: Pedido<string[]>) {
  switch (pedido.status) {
    case 'idle':
      return 'Tocá buscar'
    case 'loading':
      return 'Cargando…'
    case 'success':
      return `${pedido.data.length} ofertas` // solo acá existe data
    case 'error':
      return `Error: ${pedido.error}` // y solo acá existe error
  }
}

// Comparalo con la versión "bolsa de opcionales": permite estados imposibles
// (loading y error a la vez, data sin terminar de cargar) y obliga a revisar todo con ?.
type PedidoMalModelado = {
  loading: boolean
  error?: string
  data?: string[]
}
```

La versión "bolsa de opcionales" (`loading` , `error?` , `data?`) permite **estados imposibles**: `loading: true` con un `error` , o `data` mientras todavía carga. Con la unión discriminada esos estados ni siquiera se pueden escribir.

DEMO **Un pedido modelado como unión discriminada**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/typescript/EstadoPedido.tsx`

```tsx
import { useState } from 'react'

type Oferta = { id: number; empresa: string; puesto: string }

// Cada estado trae solo los datos que tienen sentido en ese momento.
type Pedido =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: Oferta[] }
  | { status: 'error'; error: string }

const ofertas: Oferta[] = [
  { id: 1, empresa: 'Nubelia', puesto: 'Frontend React' },
  { id: 2, empresa: 'Pagos Sur', puesto: 'Fullstack (Next.js)' },
]

function buscar(falla: boolean) {
  return new Promise<Oferta[]>((resolve, reject) =>
    setTimeout(() => (falla ? reject(new Error('El servidor no responde')) : resolve(ofertas)), 900),
  )
}

export default function EstadoPedido() {
  const [pedido, setPedido] = useState<Pedido>({ status: 'idle' })
```

```
  async function cargar(falla: boolean) {
    setPedido({ status: 'loading' })
    try {
      setPedido({ status: 'success', data: await buscar(falla)
        })
    } catch (e) {
      // En un catch, e es unknown: puede ser cualquier cosa,
        no solo un Error.
      setPedido({ status: 'error', error: e instanceof Error ?
        e.message : String(e) })
    }
  }

  return (
    <div className="stack">
      <div className="row">
        <button onClick={() => cargar(false)}>Buscar
          ofertas</button>
        <button onClick={() => cargar(true)}>Buscar (con error)
          </button>
        <button onClick={() => setPedido({ status: 'idle'
          })}>Reiniciar</button>
      </div>
      <Resultado pedido={pedido} />
      <pre className="log">{JSON.stringify(pedido, null, 2)}
        </pre>
    </div>
  )
}

function Resultado({ pedido }: { pedido: Pedido }) {
  // Dentro de cada case, TypeScript sabe qué campos existen.
  switch (pedido.status) {
    case 'idle':
      return <p className="muted">Todavía no buscaste nada.</p>
    case 'loading':
      return <p>Cargando…</p>
    case 'success':
      return (
        <ul>
          {pedido.data.map((o) => (
            <li key={o.id}>
              {o.puesto} en <strong>{o.empresa}</strong>
            </li>
          ))}
        </ul>
```

```
      )
    case 'error':
      return <p className="error-text">Error: {pedido.error}
        </p>
  }
}
```

Tocá los botones y mirá el JSON: en cada estado hay solo los campos de esa variante. En `Resultado`, cada `case` accede a `data` o `error` sin `?.` ni `!`, porque TypeScript ya sabe cuál es.

> 💡 **El error de un catch es unknown**
>
> En un `catch (e)`, `e` es `unknown`: en JavaScript se puede tirar cualquier cosa, no solo un `Error`. Por eso la demo hace `e instanceof Error ? e.message : String(e)`.

Que no se escape ningún caso

src/demos/typescript/ejemplos/narrowing/exhaustivo.ts

```
type Estado = 'postulado' | 'entrevista' | 'oferta' |
  'rechazado'

// Si un día agregás un estado nuevo, este helper hace que el
  switch deje de compilar
// hasta que lo manejes: en el default, "estado" tendría que
  ser never.
function sinCasoFaltante(valor: never): never {
  throw new Error(`Caso no manejado: ${String(valor)}`)
}

function color(estado: Estado) {
```

```
  switch (estado) {
    case 'postulado':
      return 'gris'
    case 'entrevista':
      return 'azul'
    case 'oferta':
      return 'verde'
    case 'rechazado':
      return 'rojo'
    default:
      return sinCasoFaltante(estado) // estado es never: no
        queda ninguna opción
  }
}

type EstadoNuevo = Estado | 'archivado'

function colorIncompleto(estado: EstadoNuevo) {
  switch (estado) {
    case 'postulado':
    case 'entrevista':
    case 'oferta':
    case 'rechazado':
      return 'algún color'
    default:
      // @ts-expect-error: Argument of type '"archivado"' is
        not assignable to parameter of type 'never'.
      return sinCasoFaltante(estado)
  }
}
```

En el `default`, después de descartar todos los casos, el tipo que queda es `never`. Si pasás ese valor a una función que solo acepta `never`, el día que alguien agregue una opción a la unión, **el switch deja de compilar** y te muestra exactamente dónde falta manejarla. Es una red de seguridad gratis para reducers, estados y variantes.

Type predicates y assertion functions

Cuando el chequeo es más complejo que un `typeof`, podés escribir tu propia función de narrowing:

src/demos/typescript/ejemplos/narrowing/predicados.ts

```ts
type Oferta = { id: number; empresa: string; salario?: number }

// Type predicate: una función que devuelve boolean y le
  "enseña" a TypeScript
// qué tipo tiene el valor cuando da true.
function tieneSalario(oferta: Oferta): oferta is Oferta & {
  salario: number } {
  return oferta.salario !== undefined
}

const ofertas: Oferta[] = [
  { id: 1, empresa: 'Acme', salario: 3000 },
  { id: 2, empresa: 'Globex' },
]

const conSalario = ofertas.filter(tieneSalario) // (Oferta & {
  salario: number })[]
const total = conSalario.reduce((suma, o) => suma + o.salario,
  0) // sin "?." ni "!"

// Desde TS 5.5, TypeScript infiere el predicado en casos
  simples:
const nombres = ['Acme', null, 'Globex'].filter((n) => n !==
  null) // string[]

// Assertion function: si no tira error, de ahí en adelante el
  tipo queda achicado.
function asegurarDefinido<T>(valor: T, mensaje: string):
  asserts valor is NonNullable<T> {
  if (valor == null) throw new Error(mensaje)
}

const contenedor = document.getElementById('root') //
  HTMLElement | null
asegurarDefinido(contenedor, 'Falta el div#root')
```

```
contenedor.append('listo') // HTMLElement
```

- `valor is Tipo` (**type predicate**): si la función devuelve `true`, el valor es de ese tipo. Ideal para `filter`.
- `asserts valor is Tipo` (**assertion function**): si la función no tira error, el tipo queda achicado de ahí en adelante.
- Desde TypeScript 5.5, `filter((n) => n !== null)` ya infiere el predicado solo, sin escribir nada.

satisfies y el operador !

src/demos/typescript/ejemplos/narrowing/satisfies.ts

```ts
type Estado = 'postulado' | 'entrevista' | 'oferta'

// Con una anotación, el objeto "se olvida" de sus valores exactos:
const coloresAnotados: Record<Estado, string> = {
  postulado: '#888',
  entrevista: '#08f',
  oferta: '#0a0',
}
coloresAnotados.oferta // string

// satisfies chequea que cumpla el tipo PERO conserva el tipo inferido, más preciso.
const colores = {
  postulado: '#888',
  entrevista: '#08f',
  oferta: '#0a0',
} as const satisfies Record<Estado, string>
colores.oferta // "#0a0"

// Y sigue avisando si falta algo o sobra algo:
const incompletos = {
  postulado: '#888',
  entrevista: '#08f',
```

```
  // @ts-expect-error: 'ofertta' no es una clave válida
    (¿quisiste decir 'oferta'?).
  ofertta: '#0a0',
} satisfies Record<Estado, string>

// El operador ! (non-null assertion) le dice a TS "confiá en
  mí, no es null".
// Si te equivocás, explota en runtime: preferí revisar con un
  if.
const root = document.getElementById('root')!
```

`satisfies` chequea que un valor cumpla un tipo **sin cambiar** el tipo inferido. Es ideal para objetos de configuración: tenés el chequeo (no falta ni sobra ninguna clave) y además TypeScript recuerda los valores exactos.

> **⚠️ El ! es un 'confiá en mí'**
>
> `document.getElementById('root')!` le dice a TypeScript que el valor no es `null`. No hace ningún chequeo en runtime: si te equivocás, explota igual. Usalo solo cuando estás seguro (como en `main.tsx`, con el div que está en el HTML) y preferí un `if` o una assertion function en el resto.

> **🎤 En una entrevista**
>
> "¿Cómo modelarías el estado de un fetch?" → "Con una unión discriminada por status: idle, loading, success con data y error con el mensaje. Así no existen estados imposibles como loading con error, y en cada rama del switch TypeScript sabe qué campos hay. Con un default que recibe never, si agrego un estado, el compilador me marca dónde falta manejarlo."

Capítulo 5

Funciones

Parámetros, retorno, callbacks, tipos de función, sobrecargas y void.

En React casi todo es una función: componentes, handlers, hooks, callbacks. Tiparlas bien es lo que hace que el resto se infiera solo.

Parámetros, retorno y callbacks

src/demos/typescript/ejemplos/funciones/funciones.ts

```ts
// Parámetros con tipo; el retorno se infiere (anotalo en
  funciones exportadas si querés un contrato claro).
function sumar(a: number, b: number): number {
  return a + b
}

// Opcionales (?) y valores por defecto. Los opcionales van al
  final.
function saludar(nombre: string, saludo = 'Hola', firma?:
  string) {
  return `${saludo}, ${nombre}${firma ? ` — ${firma}` : ''}`
}

// Rest parameters
function promedio(...valores: number[]) {
  return valores.reduce((a, b) => a + b, 0) / valores.length
}
```

```ts
// Parámetro objeto desestructurado: el patrón más común en
  React (así son las props).
function crearPostulacion({ empresa, remoto = true }: {
  empresa: string; remoto?: boolean }) {
  return { empresa, remoto, fecha: new Date() }
}

// Tipo de una función: (parámetros) => retorno
type Filtro = (texto: string) => boolean
const esRemoto: Filtro = (texto) => texto.includes('remoto') //
  texto se infiere como string

// Callbacks: TypeScript infiere los parámetros del contexto.
const largos = ['Acme', 'Globex'].map((empresa) =>
  empresa.length) // empresa: string

// async: siempre devuelve una Promise.
async function cargarOfertas(): Promise<string[]> {
  const respuesta = await fetch('/api/ofertas')
  return respuesta.json()
}

// @ts-expect-error: Expected 2 arguments, but got 1.
sumar(1)
```

- Los **parámetros** siempre se anotan (si no, son `any` implícito y con `strict` da error).
- El **retorno** se infiere. Anotalo cuando quieras que TypeScript te avise si la función empieza a devolver otra cosa, por ejemplo en funciones exportadas o recursivas.
- El patrón **objeto desestructurado** (`{ empresa, remoto = true }: { ... }`) es exactamente cómo se tipan las props de un componente.
- En los **callbacks** (`map`, `filter`, `onClick={(e) => ...}`) no hace falta anotar: TypeScript conoce el tipo por el contexto.
- Una función `async` siempre devuelve una `Promise<T>`.

Tipos de función

Un tipo de función se escribe como una arrow: `(texto: string) => boolean`. Lo vas a usar todo el tiempo para props que reciben callbacks:

```
type Props = {
  onSeleccionar: (id: number) => void
  filtrar?: (oferta: Oferta) => boolean
}
```

Sobrecargas y void

src/demos/typescript/ejemplos/funciones/sobrecargas.ts

```
// Overloads: varias firmas para una misma función. Se usan
  poco en código de apps;
// casi siempre una unión o un genérico lo resuelve mejor. Las
  vas a ver en tipos de librerías.
function parsear(valor: string): number
function parsear(valor: string[]): number[]
function parsear(valor: string | string[]) {
  return Array.isArray(valor) ? valor.map(Number) :
    Number(valor)
}

const uno = parsear('3') // number
const varios = parsear(['1', '2']) // number[]

// void en un tipo de callback significa "no me importa lo que
  devuelvas".
// Por eso esto compila aunque push devuelva un número:
const lista: number[] = []
const alHacerClick: () => void = () => lista.push(1)
```

Las **sobrecargas** (varias firmas para una función) existen, pero en código de apps casi siempre una unión o un genérico lo resuelven mejor. Las vas a leer en los tipos de librerías.

> ℹ️ **void no es undefined**
>
> Un callback de tipo `() => void` puede devolver cualquier cosa: `void` significa "no voy a usar lo que devuelvas". Por eso `onClick={() => lista.push(1)}` compila aunque `push` devuelva un número. Es a propósito: si no, no podrías pasar funciones que devuelven algo a donde se espera un handler.

> 🎤 **En una entrevista**
>
> "¿Cuándo anotás el tipo de retorno?" → "Los parámetros siempre; el retorno lo dejo inferir salvo en funciones exportadas o de librería, donde lo escribo como contrato para que un cambio interno no cambie la API sin que me entere."

Capítulo 6

Genéricos

Funciones y tipos que sirven para cualquier tipo sin perder información.

Un **genérico** es un parámetro, pero de tipo. Te deja escribir una función, un tipo o un componente **una vez** y que funcione con cualquier tipo, **sin perder** la información de cuál es.

Ya los usaste: `Array<string>`, `Promise<Oferta>`, `useState<number>`, `Record<string, number>`. Acá vemos cómo escribir los tuyos.

Funciones genéricas

src/demos/typescript/ejemplos/genericos/genericos.ts

```
// Un genérico es un "parámetro de tipo": la función se escribe
  una vez y sirve para cualquier T.
function primero<T>(lista: T[]): T | undefined {
  return lista[0]
}

const empresa = primero(['Acme', 'Globex']) // string |
  undefined (T se infiere: string)
const numero = primero([1, 2, 3]) // number | undefined

// Sin genérico habría que elegir entre un tipo fijo o perder
  el tipo con any/unknown.
function primeroAny(lista: any[]) {
```

```
  return lista[0] // any: se pierde el tipo
}

// Restricciones con extends: T puede ser cualquier cosa QUE
  TENGA un id.
function porId<T extends { id: number }>(lista: T[], id:
  number) {
  return lista.find((item) => item.id === id)
}

const ofertas = [{ id: 1, empresa: 'Acme', salario: 3000 }]
const oferta = porId(ofertas, 1) // conserva el tipo completo:
  { id; empresa; salario } | undefined

// @ts-expect-error: string no tiene id.
porId(['a', 'b'], 1)

// keyof y genéricos juntos: una clave que tiene que existir en
  el objeto.
function ordenarPor<T, K extends keyof T>(lista: T[], clave: K)
  {
  return [...lista].sort((a, b) => (a[clave] > b[clave] ? 1 :
    -1))
}

ordenarPor(ofertas, 'salario')
// @ts-expect-error: '"sueldo"' is not assignable to parameter
  of type '"id" | "empresa" | "salario"'.
ordenarPor(ofertas, 'sueldo')
```

- `<T>` declara el parámetro de tipo. Casi nunca lo pasás a mano: TypeScript lo **infiere** de los argumentos (`primero(['Acme'])` → `T` es `string`).
- `T extends { id: number }` es una **restricción**: T puede ser cualquier tipo, siempre que tenga un `id`. Y lo mejor: el resultado conserva el tipo completo, no solo `{ id }`.
- `K extends keyof T` es el combo clásico: una clave que tiene que existir en el objeto. Es lo que usan las funciones de "ordenar por", "agrupar por", los formularios tipados...

> 💡 **Los nombres de los genéricos**
>
> Por convención: `T` (type), `K` (key), `V` (value), `E` (element o error). Si hay varios y se confunden, usá nombres descriptivos: `TItem`, `TData`.

Tipos genéricos

src/demos/typescript/ejemplos/genericos/tipos-genericos.ts

```ts
// Los tipos e interfaces también pueden ser genéricos.
type RespuestaApi<T> = {
  data: T
  pagina: number
  total: number
}

interface Caja<T> {
  valor: T
  actualizar: (nuevo: T) => void
}

type Oferta = { id: number; empresa: string }
const respuesta: RespuestaApi<Oferta[]> = { data: [], pagina:
  1, total: 0 }

// Un tipo por defecto para el parámetro (como en las
  funciones):
type Resultado<T, E = Error> = { ok: true; valor: T } | { ok:
  false; error: E }

function dividir(a: number, b: number): Resultado<number,
  string> {
  if (b === 0) return { ok: false, error: 'No se puede dividir
    por cero' }
  return { ok: true, valor: a / b }
}

const r = dividir(10, 2)
```

```ts
if (r.ok) console.log(r.valor) // number
else console.log(r.error) // string

// Ya usaste genéricos sin darte cuenta: Array<T>, Promise<T>,
  useState<T>, Record<K, V>.
const pendiente: Promise<Oferta[]> = Promise.resolve([])
```

`RespuestaApi<T>` es el ejemplo típico: la forma de la respuesta es siempre la misma (paginación, total), lo que cambia es el tipo de `data`. Y `Resultado<T, E>` es un patrón muy usado para devolver errores sin tirar excepciones: una unión discriminada por `ok`.

Los genéricos también pueden tener un **valor por defecto** (`E = Error`), igual que los parámetros de una función.

Cuándo usar un genérico

Usá un genérico cuando **el tipo de la salida depende del tipo de la entrada**. Si la función recibe un `T` y devuelve un `T` (o un `T[]`, o un `T | undefined`), es un genérico. Si siempre devuelve lo mismo, no hace falta.

```ts
// ❌ El genérico no aporta nada: T solo aparece una vez
function imprimir<T>(valor: T): void {
  console.log(valor)
}

// ✅ Igual de flexible, más simple
function imprimir(valor: unknown): void {
  console.log(valor)
}
```

En la parte 2 usamos lo mismo para escribir componentes genéricos: una `<Lista>` o un `<Select>` que sirven para cualquier tipo de dato.

🎤 En una entrevista

"¿Para qué sirven los genéricos?" → "Para escribir código reutilizable sin perder el tipo. Por ejemplo, una función `porId<T extends { id: number }>(lista: T[])` devuelve el mismo T que recibió, con todas sus propiedades. Con any perdería el tipo y con un tipo fijo no podría reutilizarla."

Capítulo 7

Utility types y tipos calculados

Partial, Pick, Omit, ReturnType, keyof, typeof, mapped y conditional types.

TypeScript trae un set de tipos "de fábrica" que transforman otros tipos. Con ellos evitás escribir dos veces lo mismo: definís un tipo base y derivás el resto.

Los utility types del día a día

src/demos/typescript/ejemplos/utility/utility-types.ts

```
type Postulacion = {
  id: number
  empresa: string
  puesto: string
  salario: number
  notas?: string
}

// Partial: todo opcional. Ideal para un "update" que cambia
  solo algunos campos.
function actualizar(p: Postulacion, cambios:
  Partial<Postulacion>): Postulacion {
  return { ...p, ...cambios }
}
```

```ts
// Required: todo obligatorio (saca los ?).
type Completa = Required<Postulacion> // notas: string

// Readonly: nada se puede reasignar (solo el primer nivel).
type Congelada = Readonly<Postulacion>

// Pick y Omit: quedarse con algunas claves o sacar algunas.
type Resumen = Pick<Postulacion, 'empresa' | 'puesto'>
type NuevaPostulacion = Omit<Postulacion, 'id'> // el id lo
  pone el servidor

const nueva: NuevaPostulacion = { empresa: 'Acme', puesto:
  'Frontend', salario: 3000 }

// Record: objeto con claves y valores de cierto tipo.
type PorEmpresa = Record<string, Postulacion[]>

// Con uniones: Exclude, Extract y NonNullable.
type Estado = 'postulado' | 'entrevista' | 'oferta' |
  'rechazado'
type Activo = Exclude<Estado, 'rechazado'> // 'postulado' |
  'entrevista' | 'oferta'
type Final = Extract<Estado, 'oferta' | 'rechazado' | 'otro'>
  // 'oferta' | 'rechazado'
type SinNulos = NonNullable<string | null | undefined> //
  string
```

Utility type	Qué hace	Uso típico
`Partial<T>`	Todo opcional	Funciones de update, estado de un formulario a medio completar
`Required<T>`	Todo obligatorio	Config con defaults ya aplicados
`Readonly<T>`	Nada se puede reasignar	Props, estado que no se muta
`Pick<T, K>`	Solo esas claves	Una vista resumida de un tipo grande
`Omit<T, K>`	Todo menos esas claves	El "create" sin el `id` que pone el servidor

Utility type	Qué hace	Uso típico
`Record<K, V>`	Objeto con claves K y valores V	Diccionarios, mapas de estado a color
`Exclude<U, X>`	Saca opciones de una unión	Estados activos sin `'rechazado'`
`Extract<U, X>`	Se queda con las opciones que coinciden	Filtrar una unión
`NonNullable<T>`	Saca `null` y `undefined`	Después de validar

Tipos que salen de valores

Muchas veces el valor ya existe (una constante, una función, lo que devuelve un fetch) y el tipo se puede **sacar de ahí**:

`src/demos/typescript/ejemplos/utility/desde-valores.ts`

```
// Sacar tipos de cosas que ya existen, en vez de escribirlos
  dos veces.

// typeof (en posición de tipo): el tipo de una variable.
const ofertaDeEjemplo = { id: 1, empresa: 'Acme', remoto: true
  }
type Oferta = typeof ofertaDeEjemplo // { id: number; empresa:
  string; remoto: boolean }

// keyof: la unión de las claves de un tipo.
type CampoOferta = keyof Oferta // 'id' | 'empresa' | 'remoto'

// Indexed access: el tipo de una propiedad.
type Empresa = Oferta['empresa'] // string

// ReturnType y Parameters: los tipos de una función.
function crearFiltro(texto: string, soloRemoto: boolean) {
  return { texto: texto.trim(), soloRemoto, creado: Date.now()
    }
```

```ts
}
type Filtro = ReturnType<typeof crearFiltro> // { texto:
  string; soloRemoto: boolean; creado: number }
type ArgsFiltro = Parameters<typeof crearFiltro> // [texto:
  string, soloRemoto: boolean]

// Awaited: lo que hay "adentro" de una Promise.
async function cargar() {
  return [ofertaDeEjemplo]
}
type Cargadas = Awaited<ReturnType<typeof cargar>> // { id;
  empresa; remoto }[]

// El elemento de un array: [number]
const etapas = ['postulado', 'entrevista', 'oferta'] as const
type Etapa = (typeof etapas)[number] // 'postulado' |
  'entrevista' | 'oferta'
```

- `typeof valor` (en posición de tipo) da el tipo de una variable.
- `keyof T` da la unión de las claves.
- `T['prop']` (indexed access) da el tipo de una propiedad.
- `ReturnType<typeof fn>` y `Parameters<typeof fn>` dan lo que devuelve y lo que recibe una función.
- `Awaited<T>` "abre" una `Promise`.

> 💡 **Un solo lugar de verdad**
>
> Si el tipo y el valor dicen lo mismo, escribí uno y derivá el otro. Por ejemplo, la lista de etapas como `as const` y el tipo `Etapa` con `(typeof etapas)[number]`. Si alguien agrega una etapa, el tipo se actualiza solo.

Tipos calculados

Esto ya es TypeScript avanzado. No vas a escribirlo seguido, pero sí leerlo en tipos de librerías (React, Redux Toolkit, Zod, TanStack Query...):

src/demos/typescript/ejemplos/utility/avanzados.ts

```ts
// Tipos que se calculan. No los vas a escribir todos los días, pero sí leerlos en librerías.

// Mapped types: recorrer las claves de otro tipo y transformar cada una.
type Oferta = { empresa: string; salario: number; remoto: boolean }

type Banderas<T> = { [K in keyof T]: boolean }
type CamposTocados = Banderas<Oferta> // { empresa: boolean; salario: boolean; remoto: boolean }

// Así está hecho Partial en la librería estándar:
type MiPartial<T> = { [K in keyof T]?: T[K] }

// Conditional types: un "if" a nivel de tipos.
type EsArray<T> = T extends unknown[] ? 'array' : 'otro'
type A = EsArray<string[]> // 'array'
type B = EsArray<number> // 'otro'

// infer: capturar una parte del tipo mientras se compara.
type Elemento<T> = T extends (infer E)[] ? E : T
type C = Elemento<Oferta[]> // Oferta

// Template literal types: armar strings a nivel de tipos.
type Evento = 'click' | 'focus'
type Handler = `on${Capitalize<Evento>}` // 'onClick' | 'onFocus'

type Setters<T> = { [K in keyof T as `set${Capitalize<string & K>}`]: (valor: T[K]) => void }
type SettersOferta = Setters<Oferta>
// { setEmpresa: (valor: string) => void; setSalario: (valor: number) => void; setRemoto: ... }
```

- **Mapped types** (`{ [K in keyof T]: ... }`): recorren las claves de un tipo y transforman cada una. `Partial`, `Readonly` y `Record` están hechos así.
- **Conditional types** (`T extends U ? X : Y`): un "if" a nivel de tipos.
- **infer**: captura una parte del tipo mientras compara. `ReturnType` y `Awaited` están hechos con `infer`.
- **Template literal types**: arman strings a nivel de tipos. Así una librería puede saber que `setEmpresa` existe porque hay un campo `empresa`.

> 🎤 **En una entrevista**
>
> "¿Qué diferencia hay entre Pick y Omit?" → "Pick arma un tipo nuevo con las claves que le paso y Omit con todas menos esas. Uso Omit, por ejemplo, para el tipo de creación de una entidad sin el id que genera el backend: `Omit<Postulacion, 'id'>`."

Capítulo 8

Módulos, tsconfig y declaraciones

import type, por qué no enum, las opciones del tsconfig y los archivos .d.ts.

El último capítulo del lenguaje: cómo se importan y exportan los tipos, qué hace cada opción importante del `tsconfig.json`, y cómo tipar cosas que vienen de afuera.

import type

src/demos/typescript/ejemplos/modulos/imports.ts

```
// import type: importa solo tipos. Se borra al compilar, así
  que no agrega nada al bundle.
// Con "verbatimModuleSyntax" (lo trae la plantilla de Vite) es
  obligatorio para los tipos.
import type { ReactNode } from 'react'
import { useState, type Dispatch, type SetStateAction } from
  'react' // mezclado en una línea

type Props = { children: ReactNode }

// Exportar tipos es igual que exportar valores.
export type Oferta = { id: number; empresa: string }
export interface Filtros {
  texto: string
}
```

```
export function useFiltro(): [string,
  Dispatch<SetStateAction<string>>] {
  return useState('')
}
```

Con `verbatimModuleSyntax` (lo activa la plantilla de Vite), TypeScript exige marcar los imports que son **solo tipos** con `import type` o `type` adentro de las llaves. Así el compilador (y herramientas como Vite, esbuild u oxc, que procesan archivo por archivo) sabe que puede borrarlos sin mirar el resto del proyecto.

Sin enum: erasableSyntaxOnly

src/demos/typescript/ejemplos/modulos/enums.ts

```
// La plantilla de Vite activa "erasableSyntaxOnly": solo se
  permite sintaxis de TypeScript
// que se pueda borrar sin cambiar el JavaScript. Es lo que
  también exige Node.js para
// ejecutar archivos .ts directo. Los enum generan código, así
  que quedan afuera:

// @ts-expect-error: This syntax is not allowed when
  'erasableSyntaxOnly' is enabled.
enum EstadoEnum {
  Postulado,
  Entrevista,
}

// La alternativa moderna: una unión de literales...
type Estado = 'postulado' | 'entrevista' | 'oferta'

// ...o un objeto "as const" si además querés los valores en
  runtime (para listarlos, por ejemplo).
const ESTADO = {
  Postulado: 'postulado',
  Entrevista: 'entrevista',
  Oferta: 'oferta',
```

```
} as const

type EstadoDesdeObjeto = (typeof ESTADO)[keyof typeof ESTADO]
  // 'postulado' | 'entrevista' | 'oferta'

function mover(estado: EstadoDesdeObjeto) {
  return estado === ESTADO.Oferta ? '¡Felicitaciones!' :
    'Seguimos'
}
mover(ESTADO.Entrevista)
mover('oferta') // también vale el string directo
```

Casi todo TypeScript es "borrable": sacás los tipos y queda JavaScript válido. Pero hay sintaxis que **genera código**: `enum`, `namespace` y los parámetros de constructor con `public` / `private`. Con `erasableSyntaxOnly` esas construcciones dan error.

¿Por qué? Porque las herramientas modernas (Node.js ejecutando `.ts` directo, Vite, Bun) solo **borran** los tipos, no los compilan. La alternativa a `enum` es una **unión de literales** o un **objeto** `as const`, que además se lleva mejor con los strings que vienen de una API.

El tsconfig que importa

No hace falta saberse todas las opciones. Estas son las que vas a ver y tocar:

Opción	Para qué
`strict`	Todos los chequeos estrictos (`strictNullChecks`, `noImplicitAny`...). Siempre activado.
`target` / `lib`	Qué versión de JavaScript generar y qué APIs existen (`DOM`, `ES2023`...).

Opción	Para qué
`module` / `moduleResolution`	Cómo se resuelven los imports. Con Vite: `esnext` y `bundler`.
`jsx`	`react-jsx` para no tener que importar React en cada archivo.
`noEmit`	TypeScript solo chequea; Vite es quien genera el JavaScript.
`paths`	Alias de imports (`@/components/...`). Hay que configurarlos también en Vite.
`noUncheckedIndexedAccess`	`array[i]` y `obj[clave]` incluyen `undefined`. Recomendado, más estricto.
`skipLibCheck`	No revisar los `.d.ts` de `node_modules` (más rápido).

Archivos de declaración (.d.ts)

Un `.d.ts` solo tiene tipos: describe algo que existe en otro lado. Los vas a necesitar para tipar variables globales, las variables de entorno de Vite o un paquete que no trae tipos:

`src/demos/typescript/ejemplos/modulos/declaraciones.d.ts`

```
// Un archivo .d.ts solo tiene tipos: describe código que existe en otro lado.
// Este no tiene import ni export, así que todo lo que declara es global.
// (En un archivo con import/export, lo global va adentro de `declare global { ... }`.)

// Algo que agrega un script externo a window (por ejemplo, un widget de analytics).
// Las interfaces se fusionan: esto se suma a la interface Window que trae TypeScript.
interface Window {
```

```
  analytics?: { track: (evento: string, datos?: Record<string,
    unknown>) => void }
}

// Tipar las variables de entorno de Vite
  (import.meta.env.VITE_*).
interface ImportMetaEnv {
  readonly VITE_API_URL: string
}

// Un paquete de npm que no trae tipos (y no hay @types/ para
  él).
declare module 'paquete-sin-tipos' {
  export function formatear(valor: number): string
}
```

src/demos/typescript/ejemplos/modulos/usar-declaraciones.ts

```
import { formatear } from 'paquete-sin-tipos'
// Los archivos que importa Vite (imágenes, CSS, ?raw) ya
  vienen tipados por "vite/client".
import logo from '/favicon.svg'

// Gracias a declaraciones.d.ts, todo esto está tipado:
window.analytics?.track('postulacion_creada', { empresa: 'Acme'
  })

const api = import.meta.env.VITE_API_URL // string
// @ts-expect-error: Type 'string' is not assignable to type
  'number'.
const puerto: number = import.meta.env.VITE_API_URL

const texto = formatear(3000) // string
// @ts-expect-error: Argument of type 'string' is not
  assignable to parameter of type 'number'.
formatear('3000')

const src: string = logo
```

Antes de escribir un `declare module` para un paquete, buscá si existe `@types/nombre-del-paquete` (los tipos mantenidos por la comunidad en DefinitelyTyped). Hoy la mayoría de los paquetes ya traen sus tipos incluidos.

> **ℹ️ vite-env.d.ts**
>
> La plantilla de Vite incluye los tipos de `vite/client` (en `tsconfig.app.json`, en `types`). Por eso `import logo from './logo.svg'`, `import.meta.env` e `import texto from './archivo.txt?raw'` ya vienen tipados.

> **🎤 En una entrevista**
>
> "¿Por qué no usarías enum?" → "Porque genera código en runtime y no es compatible con herramientas que solo borran tipos, como Node.js o erasableSyntaxOnly. Uso una unión de literales, o un objeto as const si también necesito los valores en runtime."

Parte 2 · TypeScript en React

Cómo se tipa cada pieza de una app de React, de las props a las APIs de React 19.

Capítulo 9

Componentes y props

Props, children, ReactNode, extender elementos nativos y props discriminadas.

Un componente es una función que recibe **un objeto** (las props) y devuelve JSX. Tiparlo es tipar ese objeto. Eso es todo: no hay ninguna API especial.

Tipar las props

src/demos/typescript/ejemplos/react/props.tsx

```
import type { ReactNode } from 'react'

// Las props son un objeto: se tipan con type (o interface) y
  se desestructuran.
type TarjetaProps = {
  empresa: string
  puesto: string
  salario?: number // opcional
  etiquetas?: string[]
```

```
  children?: ReactNode // cualquier cosa que React pueda
    renderizar
}

export function Tarjeta({ empresa, puesto, salario, etiquetas =
  [], children }: TarjetaProps) {
  return (
    <article>
      <h3>
        {puesto} en {empresa}
      </h3>
      {salario !== undefined && <p>USD {salario}</p>}
      <p>{etiquetas.join(' · ')}</p>
      {children}
    </article>
  )
}

export function Lista() {
  return (
    <>
      <Tarjeta empresa="Acme" puesto="Frontend" salario={3000}>
        <button>Postularme</button>
      </Tarjeta>

      {/* @ts-expect-error: Property 'puesto' is missing. */}
      <Tarjeta empresa="Globex" />

      {/* @ts-expect-error: Type 'string' is not assignable to
        type 'number'. */}
      <Tarjeta empresa="Initech" puesto="React Dev"
        salario="3000" />
    </>
  )
}
```

- Las props se describen con un `type` (por convención, `NombreProps`) y se desestructuran en la firma.
- Las opcionales llevan `?` y el valor por defecto va en la desestructuración (`etiquetas = []`).
- Al usar el componente, TypeScript marca la prop que falta o el tipo equivocado, y el editor te autocompleta las props

disponibles.

> ⚠️ **No hace falta React.FC**
>
> Antes era común `const Tarjeta: React.FC<Props> = (...) => ...`. Hoy no se recomienda: no suma nada, complica los genéricos y en versiones viejas agregaba `children` aunque el componente no los usara. Una función con las props tipadas alcanza. El tipo de retorno se infiere, no lo anotes.

children y otros tipos de React

src/demos/typescript/ejemplos/react/children.tsx

```tsx
import type { PropsWithChildren, ReactElement, ReactNode } from
  'react'

// ReactNode: lo más amplio. JSX, strings, números, null,
  arrays… Es el tipo para children.
function Panel({ titulo, children }: { titulo: string;
  children: ReactNode }) {
  return (
    <section>
      <h2>{titulo}</h2>
      {children}
    </section>
  )
}

// PropsWithChildren<P> agrega children?: ReactNode a tus
  props.
type CardProps = PropsWithChildren<{ destacada?: boolean }>
function Card({ destacada, children }: CardProps) {
  return <div className={destacada ? 'card card--destacada' :
    'card'}>{children}</div>
}
```

```tsx
// ReactElement: solo un elemento JSX (no strings ni null).
  Útil si vas a clonarlo o leer sus props.
function Icono({ icono }: { icono: ReactElement }) {
  return <span className="icono">{icono}</span>
}

// Render prop: children como función.
function ConMouse({ children }: { children: (x: number, y:
  number) => ReactNode }) {
  return <div>{children(10, 20)}</div>
}

export function Uso() {
  return (
    <Panel titulo="Ofertas">
      <Card destacada>Texto o JSX</Card>
      <Icono icono={<svg />} />
      {/* @ts-expect-error: un string no es un ReactElement.
        */}
      <Icono icono="⭐" />
      <ConMouse>{(x, y) => `${x}, ${y}`}</ConMouse>
    </Panel>
  )
}
```

Tipo	Qué acepta	Para
`ReactNode`	Todo lo renderizable: JSX, strings, números, `null`, arrays	`children` y cualquier "slot"
`ReactElement`	Solo un elemento JSX	Cuando vas a leer o clonar el elemento
`PropsWithChildren<P>`	Tus props + `children?: ReactNode`	Atajo para layouts y wrappers
`(args) => ReactNode`	Una función que devuelve UI	Render props

Extender las props de un elemento nativo

Un `<Boton>` propio debería aceptar **todo** lo que acepta un `<button>`: `onClick`, `disabled`, `type`, `aria-label`... Escribirlas una por una es imposible. `ComponentProps<'button'>` las trae todas:

src/demos/typescript/ejemplos/react/props-nativas.tsx

```tsx
import type { ComponentProps } from 'react'

// ComponentProps<'button'> son TODAS las props de un <button> nativo:
// onClick, disabled, type, aria-*, className… y desde React 19 también ref.
type BotonProps = ComponentProps<'button'> & {
  variante?: 'primario' | 'secundario'
}

export function Boton({ variante = 'primario', className = '', ...resto }: BotonProps) {
  // El resto (onClick, disabled, children…) pasa directo al <button>.
  return <button className={`boton boton--${variante} ${className}`} {...resto} />
}

// Para sacar o reemplazar alguna prop nativa, Omit:
type InputProps = Omit<ComponentProps<'input'>, 'onChange'> & {
  onChange: (valor: string) => void // en vez del evento, te paso el texto
}

export function Input({ onChange, ...resto }: InputProps) {
  return <input {...resto} onChange={(e) => onChange(e.target.value)} />
}

// También funciona con componentes: las props de otro componente.
type PropsDelBoton = ComponentProps<typeof Boton>
```

```
export function Uso() {
  return (
    <>
      <Boton variante="secundario" onClick={() =>
        alert('hola')} disabled>
        Guardar
      </Boton>
      <Input placeholder="Empresa" onChange={(texto) =>
        console.log(texto.toUpperCase())} />
      {/* @ts-expect-error: '"peligro"' no es una variante
        válida. */}
      <Boton variante="peligro" />
    </>
  )
}
```

DEMO **Un botón con variantes que acepta todas las props de <button>**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/typescript/BotonTipado.tsx`

```
import { useState, type ComponentProps, type CSSProperties }
  from 'react'

// Todas las props de un <button> nativo, más las nuestras.
type BotonProps = ComponentProps<'button'> & {
  variante?: 'primario' | 'secundario' | 'peligro'
  compacto?: boolean
  cargando?: boolean
}

const colores = {
  primario: { background: '#087ea4', color: '#fff',
    borderColor: '#087ea4' },
  secundario: {},
  peligro: { background: '#b91c1c', color: '#fff', borderColor:
    '#b91c1c' },
```

```
} satisfies Record<NonNullable<BotonProps['variante']>,
  CSSProperties>

function Boton({
  variante = 'secundario',
  compacto = false,
  cargando = false,
  disabled,
  children,
  style,
  ...resto
}: BotonProps) {
  return (
    <button
      {...resto} // onClick, type, aria-*, title… pasan tal
        cual al <button>
      disabled={disabled || cargando}
      aria-busy={cargando}
      style={{ ...colores[variante], padding: compacto ? '2px
        8px' : undefined, ...style }}
    >
      {cargando ? 'Guardando…' : children}
    </button>
  )
}

export default function BotonTipado() {
  const [cargando, setCargando] = useState(false)
  const [clicks, setClicks] = useState(0)

  return (
    <div className="stack">
      <div className="row">
        <Boton variante="primario" onClick={() => setClicks((c)
          => c + 1)}>
          Primario ({clicks})
        </Boton>
        <Boton title="Un atributo nativo cualquiera" onClick=
          {() => setClicks((c) => c + 1)}>
          Secundario
        </Boton>
        <Boton variante="peligro" compacto onClick={() =>
          setClicks(0)}>
          Reiniciar
        </Boton>
      </div>
```

```
      <div className="row">
        <Boton
          variante="primario"
          cargando={cargando}
          onClick={() => {
            setCargando(true)
            setTimeout(() => setCargando(false), 1200)
          }}
        >
          Guardar
        </Boton>
        <span className="muted">cargando también lo
          deshabilita</span>
      </div>
    </div>
  )
}
```

El truco está en `...resto`: separás tus props (`variante`, `cargando`) y el resto pasa directo al `<button>`. Si mañana necesitás un `aria-describedby`, ya funciona.

> 💡 **ComponentProps con componentes**
>
> `ComponentProps<typeof Boton>` te da las props de **otro componente**. Sirve para envolver un componente de una librería sin tener que importar su tipo de props (que muchas veces no está exportado).

Props que dependen de otras

src/demos/typescript/ejemplos/react/props-discriminadas.tsx

```
// Props que dependen de otra prop: una unión discriminada
  evita combinaciones inválidas.
```

```tsx
type AvisoProps =
  | { tipo: 'exito'; mensaje: string }
  | { tipo: 'error'; mensaje: string; reintentar: () => void }
    // reintentar solo existe en error

export function Aviso(props: AvisoProps) {
  if (props.tipo === 'error') {
    return (
      <p role="alert">
        {props.mensaje} <button onClick=
          {props.reintentar}>Reintentar</button>
      </p>
    )
  }
  return <p>{props.mensaje}</p>
}

export function Uso() {
  return (
    <>
      <Aviso tipo="exito" mensaje="Guardado" />
      <Aviso tipo="error" mensaje="Falló" reintentar={() => {}}
        />
      {/* @ts-expect-error: falta reintentar, que es
        obligatorio cuando tipo es "error". */}
      <Aviso tipo="error" mensaje="Falló" />
    </>
  )
}

// No hace falta React.FC: una función con props tipadas
  alcanza y se lee mejor.
// (React.FC ya no agrega children implícitos, pero tampoco
  suma nada.)
```

Si `reintentar` solo tiene sentido cuando `tipo` es `'error'`, una **unión discriminada** lo expresa: TypeScript obliga a pasarlo en ese caso y no lo permite en los demás. Es mucho mejor que poner todo opcional y revisar en runtime.

🎤 En una entrevista

"¿Cómo tiparías un botón propio?" → "Con `ComponentProps<'button'>` más mis props, por ejemplo la variante. Desestructuro lo mío y paso el resto al button con spread. Así acepta onClick, disabled, aria-* y desde React 19 también ref, sin declararlos uno por uno."

Capítulo 10

Estado y eventos

useState con y sin genérico, tipos de eventos y un formulario tipado.

Los dos lugares donde más se tipa en un componente: el estado y los handlers de eventos. La buena noticia es que casi todo se infiere.

useState

src/demos/typescript/ejemplos/react/estado.tsx

```
import { useState } from 'react'

type Usuario = { id: number; nombre: string }

export function Estado() {
  // Se infiere del valor inicial: number.
  const [contador, setContador] = useState(0)

  // Cuando el inicial no alcanza (null, [] o {}), pasá el tipo
    como genérico.
  const [usuario, setUsuario] = useState<Usuario | null>(null)
  const [tags, setTags] = useState<string[]>([]) // sin
    <string[]> sería never[]

  // Uniones de literales para estados con pocas opciones.
  const [vista, setVista] = useState<'lista' | 'tablero'>
    ('lista')

  function sumar() {
```

```
    // @ts-expect-error: Argument of type 'string' is not
      assignable to parameter of type 'SetStateAction<number>'.
    setContador('1')

    // La función actualizadora recibe el valor anterior, ya
      tipado.
    setContador((anterior) => anterior + 1)
    setTags((anteriores) => [...anteriores, 'react'])
  }

  return (
    <div>
      {/* usuario puede ser null: hay que revisarlo antes de
        leer nombre */}
      <p>{usuario ? usuario.nombre : 'Sin sesión'}</p>
      <p>{usuario?.nombre ?? 'Sin sesión'}</p>
      <button onClick={() => setVista(vista === 'lista' ?
        'tablero' : 'lista')}>Vista: {vista}</button>
      <button onClick={() => setUsuario({ id: 1, nombre: 'Rick'
        })}>Entrar</button>
      <button onClick={sumar}>{contador}</button>
      <p>{tags.join(', ')}</p>
    </div>
  )
}
```

- Con un valor inicial "completo" (`0` , `'lista'` , `{ ... }`), **no anotes nada**: se infiere.
- Con un valor inicial que no alcanza (`null` , `[]` , `{}`), pasá el tipo como genérico: `useState<Usuario | null>(null)` . Sin él, `useState([])` queda como `never[]` y no vas a poder agregar nada.
- La función actualizadora (`setX(anterior => ...)`) recibe el valor anterior ya tipado.
- Si el estado puede ser `null` , TypeScript te obliga a revisarlo antes de usarlo (`usuario?.nombre` , `usuario && ...`).

Eventos

src/demos/typescript/ejemplos/react/eventos.tsx

```tsx
import type { ChangeEvent, FormEvent, KeyboardEvent, MouseEvent
  } from 'react'

export function Eventos() {
  // Inline: el tipo del evento se infiere solo, no hace falta
    escribir nada.
  const inline = <input onChange={(e) =>
    console.log(e.target.value)} />

  // Separado: hay que anotar el evento. El genérico es el
    elemento que lo dispara.
  function onChange(e: ChangeEvent<HTMLInputElement>) {
    console.log(e.target.value)
  }

  function onSelect(e: ChangeEvent<HTMLSelectElement>) {
    console.log(e.target.value)
  }

  function onClick(e: MouseEvent<HTMLButtonElement>) {
    console.log(e.clientX, e.currentTarget.textContent)
  }

  function onKeyDown(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === 'Enter') e.currentTarget.blur()
  }

  function onSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault()
    const datos = new FormData(e.currentTarget)
    const empresa = datos.get('empresa') // FormDataEntryValue
      | null: string, File o null
    console.log(String(empresa ?? ''))
  }

  // Truco: si no sabés el tipo, escribilo inline, pasá el
    mouse por "e" y copialo.
  // También podés tipar la función entera:
    ChangeEventHandler<HTMLInputElement>.
```

```
  return (
    <form onSubmit={onSubmit}>
      {inline}
      <input name="empresa" onChange={onChange} onKeyDown=
        {onKeyDown} />
      <select onChange={onSelect}>
        <option>Remoto</option>
      </select>
      {/* @ts-expect-error: el handler espera un input, no un
        botón. */}
      <button onClick={onChange}>Mal</button>
      <button onClick={onClick}>Guardar</button>
    </form>
  )
}
```

Inline, no escribas nada: en `onChange={(e) => ...}` TypeScript sabe que `e` es un `ChangeEvent<HTMLInputElement>`. Solo tenés que anotar cuando el handler es una función aparte.

Los tipos de eventos más comunes:

Evento	Tipo	Elemento típico
`onChange`	`ChangeEvent<HTMLInputElement>`	`input`, `select` (`HTMLSelectElement`), `textarea` (`HTMLTextAreaElement`)
`onSubmit`	`FormEvent<HTMLFormElement>`	`form`
`onClick`	`MouseEvent<HTMLButtonElement>`	`button`, `div`…
`onKeyDown`	`KeyboardEvent<HTMLInputElement>`	`input`
`onFocus` / `onBlur`	`FocusEvent<HTMLInputElement>`	`input`

> **⚠️ MouseEvent de React, no del DOM**
>
> Importá los tipos de eventos de `react`, no uses los globales del DOM. El `MouseEvent` global es el evento nativo; el de React es el evento sintético que recibe tu handler.

> **💡 target o currentTarget**
>
> `e.currentTarget` es el elemento que tiene el handler y está tipado con el genérico (`HTMLFormElement`). `e.target` es donde se originó el evento, que puede ser un hijo, así que en eventos de mouse TypeScript lo tipa como `EventTarget` genérico. En `onChange` de un input son el mismo elemento.

Un formulario tipado

DEMO **Un formulario con un handler genérico para todos los campos**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/typescript/FormularioTipado.tsx`

```
import { useState, type ChangeEvent, type FormEvent, type
  KeyboardEvent } from 'react'

type Modalidad = 'remoto' | 'hibrido' | 'presencial'
type Form = { empresa: string; salario: number; modalidad:
  Modalidad; destacada: boolean }
```

```
const inicial: Form = { empresa: '', salario: 2500, modalidad:
  'remoto', destacada: false }

export default function FormularioTipado() {
  const [form, setForm] = useState<Form>(inicial)
  const [log, setLog] = useState<string[]>([])

  // Un solo handler para varios campos: K es la clave y
    form[K] su tipo.
  function set<K extends keyof Form>(campo: K, valor: Form[K])
    {
    setForm((f) => ({ ...f, [campo]: valor }))
  }

  function onEmpresa(e: ChangeEvent<HTMLInputElement>) {
    set('empresa', e.target.value)
  }

  function onModalidad(e: ChangeEvent<HTMLSelectElement>) {
    // e.target.value es string: hay que convertirlo a la unión
      (acá confiamos en las <option>).
    set('modalidad', e.target.value as Modalidad)
  }

  function onKeyDown(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === 'Escape') set('empresa', '')
  }

  function onSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault()
    setLog((l) => [`Guardada: ${form.empresa || '(sin
      empresa)'} · USD ${form.salario} · ${form.modalidad}`,
      ...l])
  }

  return (
    <form className="stack" onSubmit={onSubmit}>
      <div className="row">
        <input value={form.empresa} onChange={onEmpresa}
          onKeyDown={onKeyDown} placeholder="Empresa (Esc
          borra)" />
        {/* valueAsNumber ya es number: no hace falta
          Number(e.target.value) */}
        <input
          type="number"
          value={form.salario}
```

```
          step={100}
          onChange={(e) => set('salario',
            e.target.valueAsNumber || 0)}
          style={{ width: 100 }}
        />
        <select value={form.modalidad} onChange={onModalidad}>
          <option value="remoto">Remoto</option>
          <option value="hibrido">Híbrido</option>
          <option value="presencial">Presencial</option>
        </select>
        <label className="row">
          <input type="checkbox" checked={form.destacada}
            onChange={(e) => set('destacada',
            e.target.checked)} />
          Destacada
        </label>
        <button type="submit">Guardar</button>
      </div>
      <pre className="log">{JSON.stringify(form, null, 2)}
        </pre>
      {log.length > 0 && (
        <ol className="log">
          {log.map((linea, i) => (
            <li key={i}>{linea}</li>
          ))}
        </ol>
      )}
    </form>
  )
}
```

Fijate en `set<K extends keyof Form>(campo: K, valor: Form[K])`: un solo setter para todos los campos, y TypeScript chequea que el valor coincida con el campo. `set('salario', 'mucho')` no compila.

Y en `e.target.value as Modalidad`: el `value` de un `<select>` siempre es `string`, aunque las `<option>` solo tengan valores válidos. El `as` le dice a TypeScript que confíe; si los valores vinieran de afuera, habría que validarlos.

🎤 En una entrevista

"¿Cómo tipás un handler de onChange que está separado del JSX?" → "Con `ChangeEvent<HTMLInputElement>` importado de react. Si no me acuerdo el tipo, escribo el handler inline, paso el mouse por el evento y copio el tipo que infiere TypeScript."

Capítulo 11

Refs y el DOM

useRef para nodos y valores, ref como prop y useImperativeHandle.

Los refs se tipan con el tipo del elemento del DOM (`HTMLInputElement`, `HTMLDivElement`, `HTMLVideoElement`…) o con el tipo del valor que guardan.

src/demos/typescript/ejemplos/react/refs.tsx

```
import { useEffect, useImperativeHandle, useRef, type
  ComponentProps, type Ref } from 'react'

export function Buscador() {
  // Ref al DOM: el tipo del elemento + null como valor
    inicial.
  const inputRef = useRef<HTMLInputElement>(null) //
    RefObject<HTMLInputElement | null>

  // Ref para guardar un valor mutable (un id de timer). En
    React 19 useRef pide el valor inicial.
  const timerRef = useRef<number | undefined>(undefined)

  useEffect(() => {
    // current puede ser null (antes de montar o si el elemento
      no se renderizó).
    inputRef.current?.focus()

    // @ts-expect-error: 'inputRef.current' is possibly 'null'.
    inputRef.current.focus()

    timerRef.current = window.setTimeout(() => {}, 1000)
    return () => window.clearTimeout(timerRef.current)
```

```
  }, [])

  // El tipo del ref tiene que coincidir con el elemento:
  const divRef = useRef<HTMLDivElement>(null)
  // @ts-expect-error: a un <input> no se le puede pasar un ref
    de <div> (le falta value, checked…).
  const mal = <input ref={divRef} />

  return <input ref={inputRef} />
}

// React 19: ref es una prop común. Con ComponentProps<'input'>
  ya viene incluida.
export function CampoTexto({ etiqueta, ...props }:
  ComponentProps<'input'> & { etiqueta: string }) {
  return (
    <label>
      {etiqueta} <input {...props} />
    </label>
  )
}

// O, si la declarás a mano, con el tipo Ref<T>:
export function CampoSimple({ ref }: { ref?:
  Ref<HTMLInputElement> }) {
  return <input ref={ref} />
}

// useImperativeHandle: exponer una API propia en vez del nodo
  del DOM.
export type ReproductorHandle = { play: () => void; pausa: ()
  => void }

export function Reproductor({ ref }: { ref?:
  Ref<ReproductorHandle> }) {
  const video = useRef<HTMLVideoElement>(null)
  useImperativeHandle(ref, () => ({
    play: () => void video.current?.play(),
    pausa: () => video.current?.pause(),
  }))
  return <video ref={video} src="/demo.mp4" />
}

export function Uso() {
  const reproductor = useRef<ReproductorHandle>(null)
  return (
```

```
    <>
      <Reproductor ref={reproductor} />
      <button onClick={() =>
        reproductor.current?.play()}>Play</button>
      {/* Callback ref con cleanup (React 19): la función que
        devuelve corre al desmontar. */}
      <div
        ref={(nodo) => {
          if (!nodo) return // los tipos todavía permiten null
          const observer = new ResizeObserver(() =>
            console.log(nodo.offsetWidth))
          observer.observe(nodo)
          return () => observer.disconnect()
        }}
      />
    </>
  )
}
```

Los dos usos de useRef

Para...	Cómo	Tipo de `current`
Un nodo del DOM	`useRef<HTMLInputElement>(null)`	`HTMLInputElement | null`
Un valor mutable (timer, valor anterior)	`useRef<number | undefined>(undefined)`	`number | undefined`

En los dos casos `current` es mutable. Un ref de DOM arranca en `null` y React lo completa al montar, por eso siempre hay que revisarlo (`ref.current?.focus()`).

> ℹ️ **useRef en React 19**
>
> Con los tipos de React 19, `useRef` **siempre** pide un valor inicial (`useRef<number>()` ya no compila) y desapareció la diferencia entre `RefObject` (de solo lectura) y `MutableRefObject`. Lo vemos con el resto de los cambios en React 19 con TypeScript.

ref como prop

En React 19 `ref` es una prop común, sin `forwardRef`. Hay dos formas de tiparla:

- Si tu componente extiende `ComponentProps<'input'>`, **ya está incluida**: pasa con el resto de las props.
- Si la declarás a mano, el tipo es `Ref<HTMLInputElement>` (acepta un objeto ref o un callback ref).

useImperativeHandle

Cuando el padre no necesita el nodo sino **acciones** (`play`, `pausa`, `focus`, `reset`), exponé una API propia. Definís el tipo del "handle", el hijo recibe `ref?: Ref<Handle>` y el padre crea `useRef<Handle>(null)`. TypeScript conecta las dos puntas.

Callback refs con cleanup

Desde React 19 un callback ref puede devolver una función de **cleanup**, como un effect. Los tipos todavía permiten que el nodo sea `null`, así que conviene revisarlo al principio.

🎤 En una entrevista

"¿Qué tipo le ponés a un useRef de un input?" → "`useRef<HTMLInputElement>(null)`. current es `HTMLInputElement | null`, porque hasta que el componente monta no hay nodo, así que lo uso con ?. o dentro de un effect después de revisar que exista."

Capítulo 12

Context, useReducer y custom hooks

Context sin valores falsos, acciones tipadas y hooks que devuelven tuplas.

Tres piezas que suelen ir juntas cuando el estado crece: un reducer para la lógica, un context para compartirlo y custom hooks para consumirlo con una API limpia.

Context sin valores falsos

src/demos/typescript/ejemplos/react/context.tsx

```
import { createContext, use, useState, type ReactNode } from
  'react'

type Sesion = {
  usuario: { nombre: string } | null
  entrar: (nombre: string) => void
  salir: () => void
}

// Patrón recomendado: el contexto arranca en null y un hook
  propio revisa que haya Provider.
const SesionContext = createContext<Sesion | null>(null)

export function SesionProvider({ children }: { children:
  ReactNode }) {
  const [usuario, setUsuario] = useState<Sesion['usuario']>
    (null)
```

```
  const valor: Sesion = {
    usuario,
    entrar: (nombre) => setUsuario({ nombre }),
    salir: () => setUsuario(null),
  }
  return <SesionContext value={valor}>{children}
    </SesionContext>
}

export function useSesion() {
  const sesion = use(SesionContext) // Sesion | null
  if (!sesion) throw new Error('useSesion tiene que usarse
    dentro de <SesionProvider>')
  return sesion // Sesion: los componentes no tienen que
    revisar null
}

export function BotonSalir() {
  const { usuario, salir } = useSesion()
  return usuario && <button onClick={salir}>Salir
    ({usuario.nombre})</button>
}

// La alternativa con un valor por defecto "falso" compila,
  pero esconde el error
// de olvidarte el Provider: las funciones no hacen nada y
  nadie se entera.
const ContextoConDefault = createContext<Sesion>({ usuario:
  null, entrar: () => {}, salir: () => {} })
```

El patrón recomendado:

1. `createContext<Sesion | null>(null)`: el valor por defecto es `null`, no un objeto inventado.
2. Un hook propio (`useSesion`) que lee el contexto y **tira un error claro** si no hay Provider.
3. Como después del `if` el tipo ya no incluye `null`, los componentes que usan el hook reciben un `Sesion` limpio, sin `?.` por todos lados.

Con un valor por defecto "de mentira" (`entrar: () => {}`), si te olvidás el Provider la app no se rompe: simplemente no hace nada, y te pasás un rato buscando por qué.

> ℹ️ **use(Context) o useContext**
>
> Los dos funcionan igual y tienen el mismo tipo. `use` (React 19) además se puede llamar adentro de un `if`. Y en React 19 el Provider se escribe `<SesionContext value={...}>`, sin `.Provider`.

useReducer con acciones tipadas

src/demos/typescript/ejemplos/react/reducer.tsx

```
import { useReducer } from 'react'

type Item = { id: number; empresa: string; estado: 'postulado'
  | 'entrevista' | 'oferta' }
type State = { items: Item[]; filtro: string }

// Las acciones son una unión discriminada por "type".
type Action =
  | { type: 'agregado'; item: Item }
  | { type: 'avanzado'; id: number }
  | { type: 'borrado'; id: number }
  | { type: 'filtrado'; texto: string }

const siguiente = { postulado: 'entrevista', entrevista:
  'oferta', oferta: 'oferta' } as const

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'agregado':
      return { ...state, items: [...state.items, action.item] }
        // action.item existe acá
    case 'avanzado':
      return {
```

```
        ...state,
        items: state.items.map((i) => (i.id === action.id ? {
          ...i, estado: siguiente[i.estado] } : i)),
      }
    case 'borrado':
      return { ...state, items: state.items.filter((i) => i.id
        !== action.id) }
    case 'filtrado':
      return { ...state, filtro: action.texto }
  }
}

export function Pipeline() {
  // useReducer infiere todo a partir del reducer y del estado
    inicial.
  const [state, dispatch] = useReducer(reducer, { items: [],
    filtro: '' })

  function probar() {
    dispatch({ type: 'avanzado', id: 1 })
    // @ts-expect-error: a 'avanzado' le falta el id.
    dispatch({ type: 'avanzado' })
    // @ts-expect-error: '"movido"' no es un tipo de acción.
    dispatch({ type: 'movido', id: 1 })
  }

  return (
    <button onClick={probar}>
      {state.items.length} postulaciones (filtro:
        {state.filtro})
    </button>
  )
}
```

- `Action` es una **unión discriminada por** `type`: en cada `case`, TypeScript sabe qué campos trae la acción.
- El reducer declara su retorno (`: State`), así un `case` que devuelve algo incompleto da error ahí mismo.
- `useReducer` infiere el tipo de `state` y de `dispatch` a partir del reducer: no hace falta pasar genéricos.

- Despachar una acción con un `type` inventado o sin sus campos no compila.

Para pasar `dispatch` por props o context, su tipo es `Dispatch<Action>` (importado de `react`).

Custom hooks

src/demos/typescript/ejemplos/react/custom-hooks.ts

```ts
import { useEffect, useState } from 'react'

// Un custom hook que devuelve una tupla: as const para que sea
  [T, setter] y no (T | setter)[].
export function useToggle(inicial = false) {
  const [valor, setValor] = useState(inicial)
  const alternar = () => setValor((v) => !v)
  return [valor, alternar] as const // readonly [boolean, () =>
    void]
}

// Si devuelve varias cosas, un objeto se lee mejor que una
  tupla.
export function useContador(inicial = 0) {
  const [cuenta, setCuenta] = useState(inicial)
  return { cuenta, sumar: () => setCuenta((c) => c + 1),
    reiniciar: () => setCuenta(inicial) }
}

// Genérico: el tipo del valor guardado lo decide quien usa el
  hook.
export function useLocalStorage<T>(clave: string, inicial: T) {
  const [valor, setValor] = useState<T>(() => {
    const guardado = localStorage.getItem(clave)
    return guardado ? (JSON.parse(guardado) as T) : inicial
  })

  useEffect(() => {
    localStorage.setItem(clave, JSON.stringify(valor))
  }, [clave, valor])
```

```
  return [valor, setValor] as const
}

export function useEjemplo() {
  const [abierto, alternar] = useToggle()
  const [tema, setTema] = useLocalStorage<'claro' | 'oscuro'>
    ('tema', 'claro')
  const [favoritos, setFavoritos] =
    useLocalStorage('favoritos', [] as number[])

  // @ts-expect-error: '"azul"' no es un tema válido.
  setTema('azul')

  setFavoritos((f) => [...f, 3])
  return { abierto, alternar, tema }
}
```

> ⚠️ **Tuplas: as const**
>
> Si un hook devuelve `[valor, alternar]` sin `as const`, TypeScript lo infiere como un **array** `(boolean | (() => void))[]`, y al desestructurar cada elemento es una unión inútil. Con `as const` es una tupla: `readonly [boolean, () => void]`.

- Devolvé una **tupla** cuando son dos cosas y es natural renombrarlas (como `useState`). Devolvé un **objeto** cuando son más.
- Un hook genérico como `useLocalStorage<T>` deja que quien lo usa decida el tipo. Se infiere del valor inicial, o se pasa explícito cuando el inicial no alcanza (`useLocalStorage<'claro' | 'oscuro'>(...)`).
- `JSON.parse` devuelve `any`: el `as T` del ejemplo confía en lo que haya guardado. Si el dato es importante, validalo (lo vemos en datos de APIs).

🎤 En una entrevista

"¿Cómo tipás un context?" → "Lo creo con el tipo del valor o null, y hago un hook propio que lo lee y tira error si es null. Así detecto si falta el Provider y los componentes reciben el valor ya sin null."

Capítulo 13

Componentes genéricos y patrones de props

Listas y selects que sirven para cualquier tipo, render props y slots.

Un componente es una función, así que también puede ser **genérico**. Es la forma de escribir una `<Lista>`, una `<Tabla>` o un `<Select>` que sirva para cualquier tipo de dato sin perder el tipo en los callbacks.

Un componente genérico

src/demos/typescript/ejemplos/react/genericos.tsx

```
import type { ReactNode } from 'react'

// Un componente genérico: el tipo de los items lo decide quien
  lo usa.
type ListaProps<T> = {
  items: T[]
  getKey: (item: T) => string | number
  renderItem: (item: T) => ReactNode
  vacio?: ReactNode
}

export function Lista<T>({ items, getKey, renderItem, vacio =
  'Nada por acá' }: ListaProps<T>) {
  if (items.length === 0) return <p>{vacio}</p>
  return (
```

```
    <ul>
      {items.map((item) => (
        <li key={getKey(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  )
}

type Oferta = { id: number; empresa: string; salario: number }
type Contacto = { email: string; nombre: string }

export function Uso({ ofertas, contactos }: { ofertas:
  Oferta[]; contactos: Contacto[] }) {
  return (
    <>
      {/* T se infiere como Oferta: dentro de renderItem, "o"
        tiene empresa y salario. */}
      <Lista items={ofertas} getKey={(o) => o.id} renderItem=
        {(o) => `${o.empresa}: USD ${o.salario}`} />

      {/* Y acá T es Contacto. */}
      <Lista items={contactos} getKey={(c) => c.email}
        renderItem={(c) => c.nombre} />

      {/* @ts-expect-error: Property 'salario' does not exist
        on type 'Contacto'. */}
      <Lista items={contactos} getKey={(c) => c.email}
        renderItem={(c) => c.salario} />
    </>
  )
}

// En un archivo .tsx, una arrow function genérica necesita una
  coma (<T,>)
// para que no se confunda con una etiqueta JSX:
export const Primero = <T,>({ items }: { items: T[] }) => <>
  {String(items[0])}</>
```

- `function Lista<T>(props: ListaProps<T>)`: el `T` se **infiere** de la prop `items`. No hace falta escribir `<Lista<Oferta> ...>` (aunque se puede).
- Adentro de `renderItem` y `getKey`, el parámetro ya tiene el tipo correcto: con ofertas es `Oferta`, con contactos es

`Contacto`.

- Si usás una propiedad que no existe en ese tipo, el error aparece en el JSX donde usás el componente.

> ⚠️ **Arrow functions genéricas en .tsx**
>
> En un archivo `.tsx`, `const Lista = <T>(props) => ...` no compila: el parser cree que `<T>` es una etiqueta JSX. Escribí `<T,>` (con coma) o usá `function`.

Una demo con dos tipos

DEMO **Un mismo Select genérico con ofertas y con monedas**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/typescript/ListaGenerica.tsx`

```tsx
import { useState, type ReactNode } from 'react'

// Un Select genérico: sirve para cualquier tipo de opción y
  devuelve la opción entera, no un string.
type SelectProps<T> = {
  opciones: T[]
  valor: T
  onChange: (opcion: T) => void
  getKey: (opcion: T) => string
  getLabel: (opcion: T) => string
}

function Select<T>({ opciones, valor, onChange, getKey,
  getLabel }: SelectProps<T>) {
  return (
    <select
```

```
      value={getKey(valor)}
      onChange={(e) => {
        const elegida = opciones.find((o) => getKey(o) ===
          e.target.value)
        if (elegida !== undefined) onChange(elegida)
      }}
    >
      {opciones.map((o) => (
        <option key={getKey(o)} value={getKey(o)}>
          {getLabel(o)}
        </option>
      ))}
    </select>
  )
}

function Lista<T>({ items, render }: { items: T[]; render:
  (item: T) => ReactNode }) {
  return (
    <ul>
      {items.map((item, i) => (
        <li key={i}>{render(item)}</li>
      ))}
    </ul>
  )
}

type Oferta = { id: number; empresa: string; salario: number }
type Moneda = { codigo: 'USD' | 'PYG' | 'ARS'; porDolar: number
  }

const ofertas: Oferta[] = [
  { id: 1, empresa: 'Nubelia', salario: 3200 },
  { id: 2, empresa: 'Pagos Sur', salario: 2800 },
  { id: 3, empresa: 'Andes Labs', salario: 4000 },
]
const monedas: Moneda[] = [
  { codigo: 'USD', porDolar: 1 },
  { codigo: 'PYG', porDolar: 7300 },
  { codigo: 'ARS', porDolar: 1400 },
]

export default function ListaGenerica() {
  const [oferta, setOferta] = useState(ofertas[0]) // Oferta
  const [moneda, setMoneda] = useState(monedas[0]) // Moneda
```

```
  return (
    <div className="stack">
      <div className="row">
        {/* T = Oferta */}
        <Select
          opciones={ofertas}
          valor={oferta}
          onChange={setOferta}
          getKey={(o) => String(o.id)}
          getLabel={(o) => o.empresa}
        />
        {/* T = Moneda */}
        <Select
          opciones={monedas}
          valor={moneda}
          onChange={setMoneda}
          getKey={(m) => m.codigo}
          getLabel={(m) => m.codigo}
        />
      </div>
      <p>
        {oferta.empresa} paga{' '}
        <strong>
          {(oferta.salario *
            moneda.porDolar).toLocaleString('es')}
            {moneda.codigo}
        </strong>{' '}
        por mes
        <span className="muted"> (tipos de cambio inventados)
          </span>
      </p>
      <Lista items={ofertas} render={(o) => `${o.empresa}: USD
        ${o.salario}`} />
    </div>
  )
}
```

El mismo `Select` trabaja con `Oferta` y con `Moneda`, y `onChange` devuelve **la opción entera**, no un string. Por eso `setOferta` y `setMoneda` se pueden pasar directo: TypeScript chequea que el tipo de la opción coincida con el del estado.

Patrones de props que vas a ver

Patrón	Cómo se tipa
Render prop	`render: (item: T) => ReactNode`
Slots (varias zonas de contenido)	`header?: ReactNode; footer?: ReactNode`
Envolver un elemento nativo	`ComponentProps<'input'> & { ... }`
Reemplazar una prop nativa	`Omit<ComponentProps<'input'>, 'onChange'> & { onChange: (v: string) => void }`
Variantes con datos distintos	Unión discriminada (`{ tipo: 'a'; ... } | { tipo: 'b'; ... }`)
Props de otro componente	`ComponentProps<typeof OtroComponente>`

> ### ℹ️ Componentes polimórficos
>
> Algunas librerías de UI tienen una prop `as` para cambiar el elemento (`<Texto as="h2">`). Tiparla bien es complejo (genéricos sobre `ElementType` y `ComponentPropsWithoutRef`) y rara vez lo necesitás en una app. Si te lo cruzás, sabé que existe; en tu código, preferí componentes separados o la prop `asChild` de Radix.

🎤 En una entrevista

"¿Escribiste algún componente genérico?" → "Sí, por ejemplo una lista o un select que reciben items de tipo T y callbacks como renderItem o getLabel que reciben un T. T se infiere de los items, así que el que usa el componente tiene autocompletado y chequeo de tipos en los callbacks sin escribir nada extra."

Capítulo 14

React 19 con TypeScript

React React 19

Los tipos de useActionState, useOptimistic y use, y los cambios que rompen código viejo.

Las APIs nuevas de React 19 están pensadas para que los tipos se infieran: casi nunca vas a pasar genéricos a mano.

src/demos/typescript/ejemplos/react/react19.tsx

```
import { use, useActionState, useOptimistic, type Ref } from
  'react'
import { useFormStatus } from 'react-dom'

// useActionState: el tipo del estado sale del estado inicial y
  de lo que devuelve la acción.
type EstadoForm = { error: string | null; enviado: boolean }

async function guardar(anterior: EstadoForm, datos: FormData):
  Promise<EstadoForm> {
  const empresa = datos.get('empresa')
  if (typeof empresa !== 'string' || empresa.trim() === '') {
    return { ...anterior, error: 'Falta la empresa' }
  }
  await fetch('/api/postulaciones', { method: 'POST', body:
    datos })
  return { error: null, enviado: true }
}

export function Formulario() {
  const [estado, accion, pendiente] = useActionState(guardar, {
    error: null, enviado: false })
```

```
  // estado: EstadoForm · accion: (payload: FormData) => void ·
    pendiente: boolean
  return (
    <form action={accion}>
      <input name="empresa" />
      {estado.error && <p role="alert">{estado.error}</p>}
      <BotonEnviar />
      {pendiente && <p>Guardando…</p>}
    </form>
  )
}

function BotonEnviar() {
  const { pending, data } = useFormStatus() // data: FormData |
    null
  return <button disabled={pending}>{pending ? `Enviando
    ${data?.get('empresa')}…` : 'Guardar'}</button>
}

// useOptimistic: el segundo genérico es lo que recibe la
  función que agrega el valor optimista.
type Nota = { id: string; texto: string; pendiente?: boolean }

export function Notas({ notas, agregar }: { notas: Nota[];
  agregar: (texto: string) => Promise<void> }) {
  const [optimistas, agregarOptimista] = useOptimistic<Nota[],
    string>(notas, (actuales, texto) => [
    ...actuales,
    { id: crypto.randomUUID(), texto, pendiente: true },
  ])

  async function accion(datos: FormData) {
    const texto = String(datos.get('texto'))
    agregarOptimista(texto)
    await agregar(texto)
  }

  return (
    <form action={accion}>
      {optimistas.map((n) => (
        <p key={n.id} style={{ opacity: n.pendiente ? 0.5 : 1
          }}>
          {n.texto}
        </p>
      ))}
      <input name="texto" />
```

```
    </form>
  )
}

// use(promesa): el tipo sale de la Promise.
export function Detalle({ ofertaPromise }: { ofertaPromise:
  Promise<{ empresa: string }> }) {
  const oferta = use(ofertaPromise) // { empresa: string }
  return <h2>{oferta.empresa}</h2>
}

// ref como prop (sin forwardRef):
export function Campo({ ref, ...props }: { ref?:
  Ref<HTMLInputElement>; placeholder?: string }) {
  return <input ref={ref} {...props} />
}
```

Qué infiere cada hook

Hook	De dónde sale el tipo
`useActionState(accion, inicial)`	El estado sale del inicial y del retorno de la acción. El payload, del segundo parámetro de la acción (`FormData` en un form).
`useFormStatus()`	`pending: boolean`, `data: FormData | null`, `method`, `action`.
`useOptimistic<Estado, Valor>(estado, fn)`	El estado, del primer argumento. El valor que agregás, del segundo parámetro de `fn` (conviene pasarlo como genérico).
`use(promesa)`	Lo que hay adentro de la `Promise`.
`use(Context)`	El tipo del contexto.

> 💡 **FormData siempre devuelve string | File | null**
>
> `datos.get('empresa')` es `FormDataEntryValue | null`. Revisá con `typeof valor === 'string'` antes de usarlo, o validá todo el `FormData` con Zod (siguiente capítulo).

Cambios de tipos que rompen código viejo

Al migrar a `@types/react` 19, estos son los errores que más vas a ver:

src/demos/typescript/ejemplos/react/cambios-de-tipos.tsx

```tsx
import { useRef, type JSX, type ReactElement } from 'react'

// Cambios de @types/react 19 que rompen código viejo (y cómo
  se arreglan).

export function Ejemplos() {
  // 1) useRef necesita un argumento.
  // @ts-expect-error: Expected 1 arguments, but got 0.
  const sinArgumento = useRef<number>()
  const conArgumento = useRef<number | undefined>(undefined)

  // 2) Los refs de useRef son siempre mutables (ya no existe
    la versión readonly).
  const nodo = useRef<HTMLDivElement>(null)

  return (
    <>
      {/* 3) Un callback ref solo puede devolver una función de
        cleanup. Las arrow
          functions de una sola expresión que devuelven algo
            ahora dan error: */}
      {/* @ts-expect-error: la asignación devuelve el nodo, que
        no es un cleanup válido. */}
```

```
      <div ref={(n) => (nodo.current = n)} />
      <div
        ref={(n) => {
          nodo.current = n
        }}
      />
    </>
  )
}

// 4) El namespace global JSX ya no existe: se importa el de
  React.
type Etiqueta = keyof JSX.IntrinsicElements // 'div' | 'span' |
  'button' | ...

// 5) Las props de un ReactElement ahora son unknown (antes
  any): hay que decir el tipo.
function leerTitulo(elemento: ReactElement<{ title: string }>)
  {
  return elemento.props.title
}

function leerSinTipo(elemento: ReactElement) {
  // @ts-expect-error: 'elemento.props' is of type 'unknown'.
  return elemento.props.title
}
```

Error	Arreglo
`useRef` sin argumento	`useRef<T>(null)` o `useRef<T | undefined>(undefined)`
`MutableRefObject` deprecado	Usá `RefObject`: ahora todos los refs son mutables
`JSX` no encontrado	`import type { JSX } from 'react'` (el namespace global se fue)
`props` de un `ReactElement` es `unknown`	Pasá el tipo: `ReactElement<{ title: string }>`
Callback ref que devuelve algo	Usá llaves: `ref={(n) => { miRef.current = n }}`

Error	Arreglo
`forwardRef` deprecado	`ref` como prop común (`Ref<T>` o `ComponentProps<'elemento'>`)
`propTypes` y `defaultProps` en funciones	Se ignoran: usá TypeScript y valores por defecto en la desestructuración

Hay un codemod que arregla la mayoría automáticamente: `npx types-react-codemod@latest preset-19 ./src`.

> **🎤 En una entrevista**
>
> "¿Qué cambió en los tipos con React 19?" → "useRef pide siempre un argumento y todos los refs son mutables, el namespace JSX ahora se importa de react, las props de ReactElement son unknown en vez de any y ref pasa a ser una prop común, así que forwardRef ya no hace falta."

Capítulo 15

Datos de APIs: unknown y Zod

Por qué as no alcanza y cómo validar en el borde con un esquema que también es el tipo.

TypeScript chequea tu código, pero **no puede chequear lo que llega por la red**. `response.json()` devuelve `any`, y cualquier tipo que le pongas es una promesa que hacés vos, no algo comprobado. Este capítulo es sobre cómo cerrar ese hueco.

El problema del as

src/demos/typescript/ejemplos/react/apis.ts

```
import { z } from 'zod'

// response.json() devuelve Promise<any>. Anotarlo con "as" no
  valida nada:
type Oferta = { id: number; empresa: string; salario: number }

async function cargarConfiando(): Promise<Oferta[]> {
  const res = await fetch('/api/ofertas')
  return (await res.json()) as Oferta[] // si el backend
    cambia, el error aparece lejos de acá
}

// Mejor: tratarlo como unknown y validarlo en el borde. Con
  Zod, el esquema ES el tipo.
```

```ts
const OfertaSchema = z.object({
  id: z.number(),
  empresa: z.string().min(1),
  salario: z.number().positive().optional(),
  modalidad: z.enum(['remoto', 'hibrido', 'presencial']),
})

// El tipo se deriva del esquema: no hay que escribirlo dos
  veces.
type OfertaValidada = z.infer<typeof OfertaSchema>
// { id: number; empresa: string; salario?: number | undefined;
  modalidad: 'remoto' | 'hibrido' | 'presencial' }

export async function cargarOfertas():
  Promise<OfertaValidada[]> {
  const res = await fetch('/api/ofertas')
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  const json: unknown = await res.json()
  return z.array(OfertaSchema).parse(json) // tira un error
    claro si algo no coincide
}

// safeParse no tira: devuelve una unión discriminada por
  success.
export function validar(json: unknown) {
  const resultado = OfertaSchema.safeParse(json)
  if (!resultado.success) {
    return resultado.error.issues.map((i) =>
      `${i.path.join('.')}: ${i.message}`)
  }
  return resultado.data.empresa // OfertaValidada
}
```

`(await res.json()) as Oferta[]` compila siempre. Si el backend cambia `salario` por `sueldo`, o manda el `id` como string, TypeScript no se entera: el error aparece mucho después, en un componente que muestra `undefined` o en un cálculo que da `NaN`.

La regla: **los datos externos entran como `unknown` y se validan en el borde** (donde los recibís), así el resto de la app puede confiar en los tipos.

Validar con Zod

Zod es la librería de validación más usada en el ecosistema de React y Next.js. Definís un **esquema**, y de ese esquema salen dos cosas:

- La **validación en runtime**: `parse` (tira un error) o `safeParse` (devuelve una unión discriminada por `success`).
- El **tipo de TypeScript**, con `z.infer<typeof Schema>`. No escribís el tipo dos veces y nunca se desincronizan.

DEMO **Validar una respuesta de API con Zod**

▶ Demo interactiva: corré `npm run dev` y abrí este capítulo para probarla.

`src/demos/typescript/ValidarRespuesta.tsx`

```
import { useState } from 'react'
import { z } from 'zod'

// El esquema describe y valida la respuesta. El tipo Oferta
  sale de acá.
const OfertaSchema = z.object({
  id: z.number(),
  empresa: z.string().min(1, 'no puede estar vacía'),
  salario: z.number().positive().optional(),
  modalidad: z.enum(['remoto', 'hibrido', 'presencial']),
  tags: z.array(z.string()).default([]),
})
type Oferta = z.infer<typeof OfertaSchema>

const ejemplos = {
  valida: '{\n  "id": 1,\n  "empresa": "Nubelia",\n  "salario":
    3200,\n  "modalidad": "remoto"\n}',
  rota: '{\n  "id": "1",\n  "empresa": "",\n  "modalidad":
    "full remote",\n  "tags": ["react", 3]\n}',
}
```

```
type Resultado = { ok: true; oferta: Oferta } | { ok: false;
  errores: string[] }

function validar(texto: string): Resultado {
  let json: unknown // JSON.parse devuelve any: lo bajamos a
    unknown a propósito
  try {
    json = JSON.parse(texto)
  } catch {
    return { ok: false, errores: ['No es JSON válido'] }
  }
  const r = OfertaSchema.safeParse(json)
  return r.success
    ? { ok: true, oferta: r.data }
    : { ok: false, errores: r.error.issues.map((i) =>
      `${i.path.join('.') || '(raíz)'}: ${i.message}`) }
}

export default function ValidarRespuesta() {
  const [texto, setTexto] = useState(ejemplos.valida)
  const resultado = validar(texto) // estado derivado: se
    calcula en cada render

  return (
    <div className="stack">
      <div className="row">
        <button onClick={() =>
          setTexto(ejemplos.valida)}>Respuesta válida</button>
        <button onClick={() =>
          setTexto(ejemplos.rota)}>Respuesta rota</button>
      </div>
      <textarea
        value={texto}
        onChange={(e) => setTexto(e.target.value)}
        rows={7}
        spellCheck={false}
        style={{ fontFamily: 'var(--font-mono)', fontSize:
          '0.85rem', width: '100%' }}
        aria-label="JSON de la respuesta"
      />
      {resultado.ok ? (
        <p className="ok-text">
          ✓ Válida: {resultado.oferta.empresa},
            {resultado.oferta.modalidad}
```

```
          {resultado.oferta.salario !== undefined && `, USD
            ${resultado.oferta.salario}`} · tags: [
          {resultado.oferta.tags.join(', ')}]
        </p>
      ) : (
        <ul className="error-text">
          {resultado.errores.map((e) => (
            <li key={e}>{e}</li>
          ))}
        </ul>
      )}
    </div>
  )
}
```

Editá el JSON: cambiá `"remoto"` por `"full remote"`, borrá la empresa o poné el `id` entre comillas. Cada error dice el campo y el problema. Fijate también que `tags` tiene `.default([])`: si no viene, el resultado igual es un array.

Dónde validar

Dato	Dónde
Respuesta de una API	En la función que hace el fetch, antes de devolverla
`FormData` de un formulario	En la acción (o Server Function) que lo recibe
`localStorage`, query params, `postMessage`	Al leerlos
Variables de entorno	Al arrancar la app

Así los componentes reciben datos ya tipados y validados, y no tienen que defenderse de nada.

💡 El mismo esquema en el front y el back

En un proyecto con Next.js (o con un backend en Node con TypeScript), el mismo esquema de Zod valida el formulario en el cliente y el body en el servidor. Un solo lugar donde vive la forma de los datos.

ℹ️ Otras opciones

Valibot y ArkType hacen lo mismo con bundles más chicos. Si el backend publica un esquema OpenAPI, también podés generar los tipos con herramientas como `openapi-typescript`. Lo importante es la idea: **no confiar en `as` para datos externos**.

🎤 En una entrevista

"¿Cómo tipás la respuesta de una API?" → "La trato como unknown y la valido en el borde con un esquema de Zod. Del esquema saco el tipo con z.infer, así la validación en runtime y el tipo nunca se desincronizan. Un as solo le dice a TypeScript que confíe, no comprueba nada."

Parte 3 · Repaso

Todo en una página y las preguntas que te pueden hacer en una entrevista.

Capítulo 16

Chuleta y preguntas de entrevista

Los patrones de la guía en snippets cortos y respuestas para entrevistas.

Todo lo de la guía en una página, para repasar antes de una entrevista o tener a mano mientras codeás.

Tipos

```
let a: string | null = null                // unión con null
const estados = ['a', 'b'] as const        // readonly ['a',
  'b']
type Estado = (typeof estados)[number]     // 'a' | 'b'
type Props = { id: number; nombre?: string; readonly creado:
  Date }
type Mapa = Record<Estado, string>         // obliga a
  completar todas las claves
const config = { a: 1 } satisfies Record<string, number> //
  chequea sin perder el tipo
function nunca(x: never): never { throw new Error(String(x)) }
  // switch exhaustivo
```

Utility types

Necesito...	Uso
Un update parcial	`Partial<T>`
Crear sin el id	`Omit<T, 'id'>`
Algunas claves	`Pick<T, 'a' | 'b'>`
Sacar opciones de una unión	`Exclude<U, 'x'>`
Lo que devuelve una función async	`Awaited<ReturnType<typeof fn>>`
Las claves como unión	`keyof T`
El tipo de una propiedad	`T['prop']`

React

```
// Props
type Props = ComponentProps<'button'> & { variante?: 'primario'
  | 'secundario' }
function Boton({ variante = 'primario', ...resto }: Props) { /*
  ... */ }

// children
{ children: ReactNode }

// Estado
const [usuario, setUsuario] = useState<Usuario | null>(null)
const [tags, setTags] = useState<string[]>([])

// Eventos (solo si el handler está separado)
function onChange(e: ChangeEvent<HTMLInputElement>) {}
function onSubmit(e: FormEvent<HTMLFormElement>) {}

// Refs
const input = useRef<HTMLInputElement>(null)
function Campo({ ref }: { ref?: Ref<HTMLInputElement> }) {}

// Context
const Ctx = createContext<Valor | null>(null)
```

```
function useValor() {
  const v = use(Ctx)
  if (!v) throw new Error('Falta el Provider')
  return v
}

// Reducer
type Action = { type: 'agregado'; item: Item } | { type:
  'borrado'; id: number }

// Hook que devuelve una tupla
return [valor, setValor] as const

// Componente genérico
function Lista<T>({ items, render }: { items: T[]; render:
  (item: T) => ReactNode }) {}

// Datos externos
const Oferta = z.object({ id: z.number(), empresa: z.string()
  })
type Oferta = z.infer<typeof Oferta>
```

Preguntas de entrevista

¿any o unknown? Los dos aceptan cualquier valor. `any` apaga el chequeo; `unknown` obliga a revisarlo antes de usarlo. Para datos externos, `unknown` y validación.

¿type o interface? Para objetos, casi lo mismo. `type` sirve también para uniones y tipos calculados; `interface` se extiende con `extends` y se fusiona si se declara dos veces. Elegí uno y sé consistente.

¿Qué es una unión discriminada? Una unión de objetos con una propiedad literal en común (`status`, `type`). Chequeando esa propiedad, TypeScript sabe qué variante es y qué campos tiene. Es la forma de modelar estados sin combinaciones imposibles.

¿Qué es never y para qué sirve? El tipo de lo que no puede pasar. Sirve para chequeos exhaustivos: si en el `default` de un switch el valor no es `never`, falta manejar un caso.

¿Para qué sirven los genéricos? Para reutilizar código sin perder el tipo: la salida depende de la entrada (`primero<T>(lista: T[]): T | undefined`).

¿Qué hace satisfies? Chequea que un valor cumpla un tipo sin reemplazar el tipo inferido, que suele ser más preciso.

¿Por qué no enum? Genera código en runtime y no es compatible con herramientas que solo borran tipos (`erasableSyntaxOnly`, Node.js). Mejor una unión de literales o un objeto `as const`.

¿Cómo tipás children? Con `ReactNode`, que acepta todo lo renderizable.

¿React.FC sí o no? No hace falta: una función con las props tipadas alcanza y se lleva mejor con los genéricos.

¿Cómo tipás un evento? Inline se infiere. Separado: `ChangeEvent<HTMLInputElement>`, `FormEvent<HTMLFormElement>`, `MouseEvent<HTMLButtonElement>`, importados de `react`.

¿Cómo extendés las props de un elemento nativo? `ComponentProps<'button'> & MisProps`, desestructurando lo propio y pasando `...resto` al elemento.

¿Qué pasa con los datos de una API? TypeScript no los puede chequear: `res.json()` es `any`. Se validan en el borde con Zod y el tipo sale del esquema con `z.infer`.

¿Qué es el `!` después de una expresión? Non-null assertion: le dice a TypeScript que no es `null` sin chequear nada. Usalo poco; preferí un `if`.

¿as es lo mismo que validar? No. `as` es una afirmación tuya que TypeScript acepta sin comprobar. Solo validar en runtime (con un `if` o un esquema) da garantías.